

# **FINAL PROJECT ASSIGNMENT REPORT**

## **Brain Computer Interface (BCI) Motor Imagery Analysis and EEG Feature Extraction Using Machine Learning**



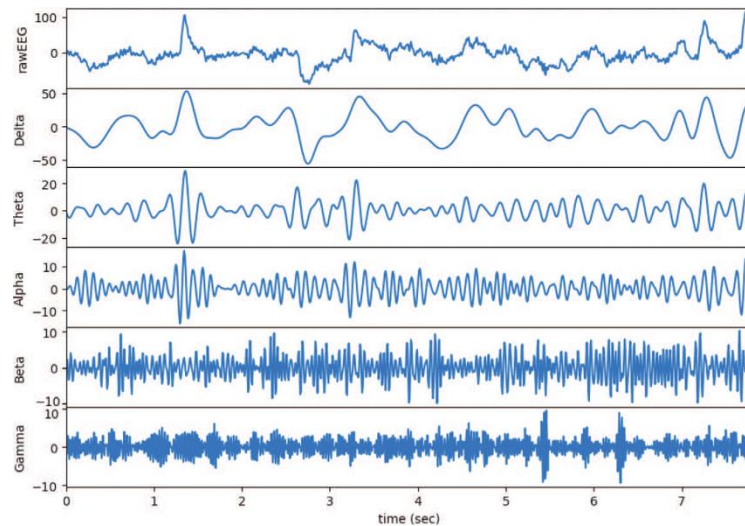
Name : Jeremia Christ Immanuel Manalu  
NRP : 5023231017  
Course : Non-Stationary Signal Analysis  
Class : A  
Lecturer : Muhammad Adib Syamlan, S.T., M.T.  
Department : Biomedical Engineering

**FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS  
TECHNOLOGY  
INSTITUT TEKNOLOGI SEPULUH NOPEMBER  
2025**

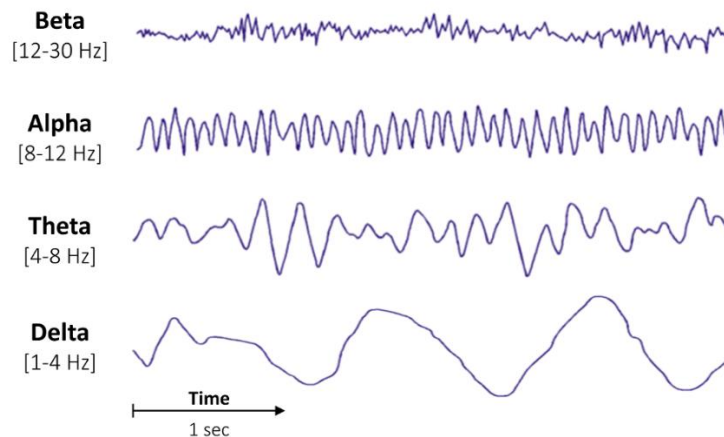
# CHAPTER I. FUNDAMENTAL THEORY

## 1.1. EEG Signal

Electroencephalography (EEG) is a non-invasive monitoring method used to record the electrical activity of the brain. It detects voltage fluctuations resulting from ionic current within the neurons of the brain. In the context of Brain-Computer Interfaces (BCI), a specific paradigm known as Motor Imagery (MI) is frequently utilized. MI involves the mental rehearsal of a motor act without any overt motor output. Research indicates that the imagination of movement activates the sensorimotor cortex in a manner similar to actual movement execution. This activation results in specific oscillatory patterns, particularly in the mu (8–13 Hz) and beta (13–30 Hz) frequency bands, which can be detected via electrodes placed over the motor cortex (positions C3, Cz, and C4 according to the international 10-20 system). These neural signatures provide the control signals necessary for BCI systems to discriminate between different cognitive tasks, such as imagining left-hand versus right-hand movements. The example of the EEG signal can be seen in **Figure 1.1** and for the frequency bands of the EEG signal can be seen in **Figure 1.2**.



**Figure 1.1.** An example of EEG Signal after Preprocessing



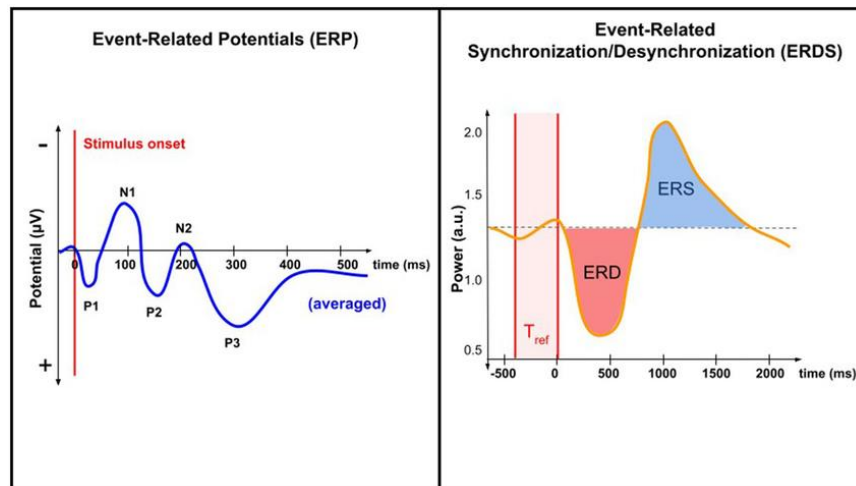
**Figure 1.2.** The Frequency Bands of the EEG Signal

## 1.2. Event-Related Desynchronization and Synchronization (ERD/ERS)

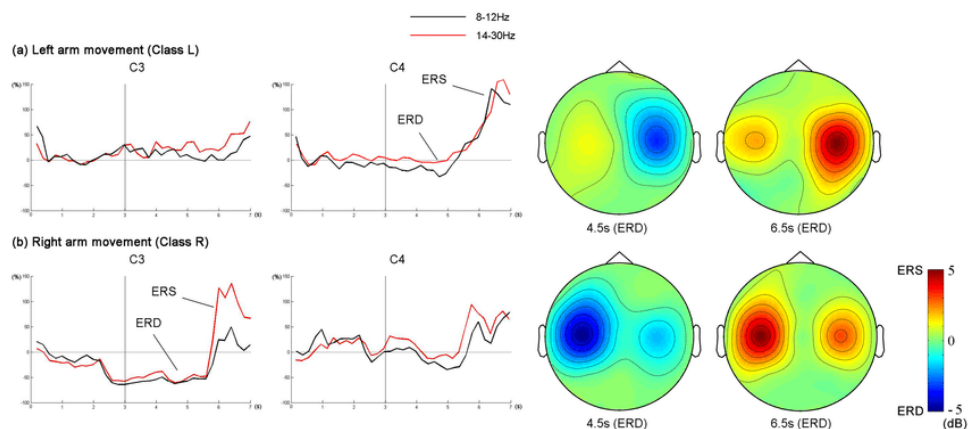
The neurophysiological basis for detecting Motor Imagery lies in the phenomena of Event-Related Desynchronization (ERD) and Event-Related Synchronization (ERS). These terms describe the changes in the ongoing EEG power within specific frequency bands relative to a reference interval. ERD represents a decrease in the power of functional cortical networks, signifying neuronal activation and increased cortical excitability. Conversely, ERS represents an increase in power, signifying neuronal idling or inhibition. During hand motor imagery, a characteristic contralateral ERD is typically observed; for instance, imagining a right-hand movement induces desynchronization in the left hemisphere (C3), while the ipsilateral side may show ERS or remain neutral. Quantitatively, ERD/ERS is expressed as a percentage change in power ( $A$ ) relative to a baseline reference ( $R$ ):

$$ERD\% = \frac{A - R}{R} \times 100$$

A negative value indicates ERD (activation), while a positive value indicates ERS (recovery or idling). This distinction is critical for identifying the "intention" of movement in EEG traces. The example of ERD/S on the EEG signal can be seen in both **Figure 1.3** and **Figure 1.4** below.



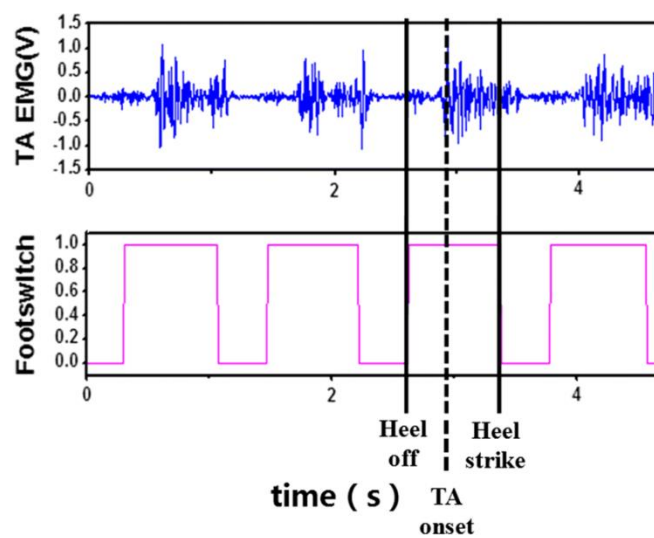
**Figure 1.3.** ERP and ERD/ERS examples



**Figure 1.4.** ERD/ERS curves (left side) and grand average topographies (right side) of left arm movement (a) and right arm movement (b)

### 1.3. Signal Preprocessing and Time-Frequency Analysis

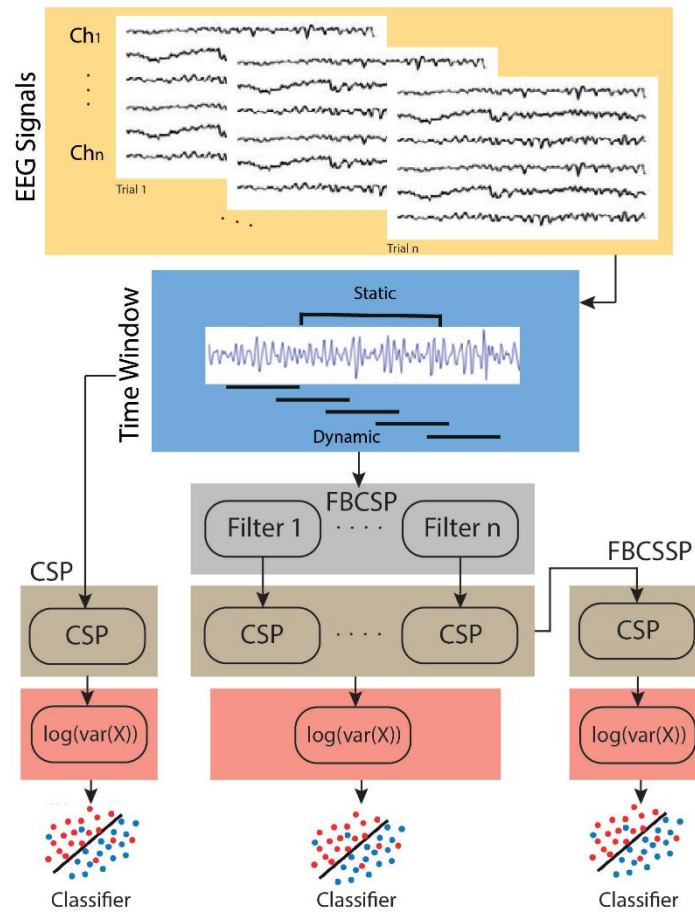
Raw EEG signals are inherently non-stationary and contaminated by various noise sources, including power line interference, electromyogram (EMG) artifacts, and ocular movements (EOG). To isolate relevant brain rhythms, bandpass filtering is essential, typically utilizing Infinite Impulse Response (IIR) filters like the Butterworth filter to extract the 8–30 Hz range covering mu and beta bands. Furthermore, because EEG spectral content changes over time, time-frequency analysis methods such as the Continuous Wavelet Transform (CWT) are employed. Unlike the standard Fourier Transform which provides only frequency information, CWT convolves the signal with a mother wavelet (for example, Morlet or Mexican Hat) to construct a detailed spectrogram. This allows for the precise temporal localization of ERD and ERS onsets, providing a visual and analytical basis for selecting the optimal time windows for classification.



*Figure 1.3. Example of A Segmentation between Foot Switch to EMG Signal*

### 1.4. Feature Extraction: Common Spatial Pattern (CSP) and Temporal Statistics

The high dimensionality and volume conduction effects of EEG data necessitate robust feature extraction techniques. The Common Spatial Pattern (CSP) algorithm is the gold standard for two-class Motor Imagery classification. CSP constructs spatial filters that maximize the variance of the spatially filtered signal for one class (for example, Left Hand) while minimizing it for the other (for example, Right Hand). Mathematically, this involves the simultaneous diagonalization of the covariance matrices of both classes. The log-transformed variance of these spatially filtered signals serves as a highly discriminative feature. In addition to spatial features, temporal statistical moments, such as Mean, Variance, Skewness, and Kurtosis, provide complementary information regarding the amplitude distribution and signal complexity within the active motor imagery window, enhancing the separability of the classes.



**Figure 1.5.** An example of a block diagram of the CSP-based methods (CSP, FBCSP, and FBCSSP)

### 1.5. Machine Learning Classification

The final stage of a BCI pipeline involves mapping the extracted features to specific class labels. Machine Learning (ML) algorithms simulate the decision boundary between cognitive states. Linear classifiers, such as Linear Support Vector Machines (SVM) and Logistic Regression, are often favored for their robustness against overfitting in high-dimensional spaces. However, non-linear dependencies in EEG data may require more complex models like Kernel SVM (using Radial Basis Functions), Random Forests (an ensemble of decision trees), or Multi-Layer Perceptrons (MLP) within the domain of Artificial Neural Networks. The performance of these classifiers is rigorously evaluated using metrics such as Accuracy, Precision, Recall, and the F1-Score, typically employing cross-validation or separate testing datasets to ensure the model generalizes well to unseen neural data.

## CHAPTER II. RESULT AND ANALYSIS

### 2.1. Problems Statement

Silahkan dikerjakan dan dikumpulkan programnya + laporannya. Terima kasih.

**Update:** EAS --> Project.

- Lakukan analisa ERP (ERD dan ERS) pada 3 channel EEG [dataset](#) BCI Competition IV 2B (sudah dijelaskan di kelas). Kemudian lakukan ekstraksi fitur temporal (time domain), misalkan mean, var, dll.
- Lakukan Common Spatial Pattern (CSP), metode umum yang digunakan untuk MI EEG. Gunakan 3 channel saja (C3, Cz, dan C4), tidak perlu gunakan channel EoG. Hasil berupa fitur.
- Kombinasikan fitur ERP dengan CSP untuk proses training, klasifikasi 2 kelas Right Hand (RH) dan Left Hand (LH) dengan Machine Learning (bebas misalkan SVM, RF, MLP, dll.). Plot confusion matrix dan print performansi dari Machine Learning (akurasi = wajib).

### 2.2. Code Explanation

This section will explain the practical implementation of the EEG Motor Imagery Analysis Framework, that is implemented as a modular Python application leveraging high-performance C++ libraries. This architecture ensures efficient signal processing while providing a flexible and user-friendly Graphical User Interface (GUI). The pipeline progresses through data management, time-frequency analysis, preprocessing, feature extraction, and finally, machine learning classification and inference. Each step is designed to be transparent, with visual feedback and descriptive explanations.

*For brevity purposes, the code lines snippet will not be showed fully, because each modules have many lines of code, and the GUI script is intentionally omitted. So, in the below explanation I will only showed the main class or main function on each script.*

#### a. *eeg\_core.cpp*

This C++ file serves as the high-performance backend, implementing computationally intensive digital signal processing algorithms. It is compiled into a Dynamic Link Library (DLL) or shared object (.dll on Windows, .so on Linux) to be called from Python via ctypes.

```
1. DLLEXPORT void apply_filter(double* input, int length,
2.                             double* b, int b_len,
3.                             double* a, int a_len,
4.                             double* output) {
5.
6.     // Initialize output with zeros to avoid garbage values
7.     for (int i = 0; i < length; i++) {
8.         output[i] = 0.0;
9.     }
10.
11.    // Direct Form I Implementation (Stable and simple)
```

```

12. // Formula:  $y[n] = (b_0 \cdot x[n] + \dots + b_M \cdot x[n-M]) - (a_1 \cdot y[n-1] + \dots + a_N \cdot y[n-N])$ 
13.
14. for (int n = 0; n < length; n++) {
15.     // Feedforward part (b coefficients)
16.     for (int k = 0; k < b_len; k++) {
17.         if (n - k >= 0) {
18.             output[n] += b[k] * input[n - k];
19.         }
20.     }
21.
22.     // Feedback part (a coefficients)
23.     for (int k = 1; k < a_len; k++) {
24.         if (n - k >= 0) {
25.             output[n] -= a[k] * output[n - k];
26.         }
27.     }
28.
29.     // Normalize by a[0] if it's not 1.0 (Safety check)
30.     if (a[0] != 1.0 && a[0] != 0.0) {
31.         output[n] /= a[0];
32.     }
33. }
34. }

```

The `apply_filter` function implements a Direct Form I Infinite Impulse Response (IIR) filter. It takes a 1D input signal, numerator coefficients `b`, and denominator coefficients `a`. The core of the filter is a difference equation that calculates each output sample `output[n]` based on current and past input samples (`input[n-k]`) and past output samples (`output[n-k]`). This is used for efficient bandpass filtering.

```

1. DLLEXPORT void moving_average(double* input, int length, int window_size,
2.     double* output) {
3.
4.     double sum = 0.0;
5.
6.     // Initial window
7.     for (int i = 0; i < length; i++) {
8.         sum += input[i];
9.
10.        if (i >= window_size) {
11.            sum -= input[i - window_size];
12.        }
13.
14.        if (i >= window_size - 1) {
15.            output[i] = sum / window_size;
16.        } else {

```

```

17.         // Padding strategy: Average of available samples (ramp-up)
18.         output[i] = sum / (i + 1);
19.     }
20. }
21. }

```

The `moving_average` function computes a simple sliding-window average. For each point in the input signal, it calculates the mean of `window_size` samples surrounding (or preceding) it. This effectively smooths out high-frequency fluctuations, extracting the slower-varying envelope of the signal.

```

1. DLLEXPORT void compute_cwt_magnitude(double* input, int length,
2.                                     double* scales, int num_scales,
3.                                     double fs,
4.                                     double* output) {
5.
6.     double omega0 = 6.0; // Standard central frequency for Morlet
7.
8.     for (int s = 0; s < num_scales; s++) {
9.         double scale = scales[s];
10.
11.        // Define half-width of the wavelet in samples (Support)
12.        int half_width = (int)(scale * 4.0);
13.        int kernel_len = 2 * half_width + 1;
14.
15.        std::vector<std::complex<double>> kernel(kernel_len);
16.
17.        double norm_factor = pow(PI, -0.25) / sqrt(scale);
18.
19.        for (int k = 0; k < kernel_len; k++) {
20.            double t = (k - half_width) / fs; // Time in seconds
21.            double t_scaled = t / (scale / fs); // Normalized time
22.
23.            // Complex Morlet: exp(i * w0 * t_scaled) * exp(-0.5 * t_scaled^2)
24.            double gaussian = exp(-0.5 * t_scaled * t_scaled);
25.            double real_part = cos(omega0 * t_scaled) * gaussian;
26.            double imag_part = sin(omega0 * t_scaled) * gaussian;
27.
28.            kernel[k] = std::complex<double>(real_part, imag_part) * norm_factor;
29.        }
30.
31.        // Convolution loop
32.        for (int t = 0; t < length; t++) {
33.            std::complex<double> conv_sum(0.0, 0.0);
34.
35.            for (int k = 0; k < kernel_len; k++) {
36.                int input_idx = t - half_width + k;
37.

```



```

38.         if (input_idx >= 0 && input_idx < length) {
39.             conv_sum += input[input_idx] * std::conj(kernel[k]);
40.         }
41.     }
42.     output[s * length + t] = std::abs(conv_sum);
43. }
44. }
45. }

```

The `compute_cwt_magnitude` function performs the Continuous Wavelet Transform (CWT) using a complex Morlet wavelet. It takes a 1D signal and an array of scales (corresponding to frequencies). For each scale, it generates a Morlet wavelet kernel and performs time-domain convolution with the input signal. The magnitude of the complex convolution result is stored in the output array, forming a time-frequency representation.

```

1. DLLEXPORT void compute_cwt_mexican_hat(double* input, int length,
2.                                         double* scales, int num_scales,
3.                                         double fs,
4.                                         double* output) {
5.
6.     for (int s = 0; s < num_scales; s++) {
7.         double scale = scales[s];
8.
9.         // Support for Mexican Hat is typically [-5, 5] standard deviations
10.        int half_width = (int)(scale * 5.0);
11.        int kernel_len = 2 * half_width + 1;
12.
13.        std::vector<double> kernel(kernel_len);
14.
15.        // Normalization factor A = 2 / (sqrt(3*scale) * pi^0.25)
16.        double norm_factor = 2.0 / (sqrt(3.0 * scale) * pow(PI, 0.25));
17.
18.        for (int k = 0; k < kernel_len; k++) {
19.            double t = (k - half_width) / fs;
20.            double t_sq = (t * t) / ((scale/fs) * (scale/fs)); // t^2 / sigma^2
21.
22.            // Mexican Hat Function
23.            kernel[k] = norm_factor * (1.0 - t_sq) * exp(-0.5 * t_sq);
24.        }
25.
26.        // Convolution Loop (Real-valued)
27.        for (int t = 0; t < length; t++) {
28.            double conv_sum = 0.0;
29.
30.            for (int k = 0; k < kernel_len; k++) {
31.                int input_idx = t - half_width + k;
32.                if (input_idx >= 0 && input_idx < length) {

```

```

33.         conv_sum += input[input_idx] * kernel[k];
34.     }
35. }
36. // Store Magnitude (Absolute value since Mexican Hat can be negative)
37. output[s * length + t] = std::abs(conv_sum);
38. }
39. }
40. }

```

The `compute_cwt_mexican_hat` function is similar to `compute_cwt_magnitude` but uses a real-valued Mexican Hat (Ricker) wavelet. This wavelet is a second derivative of a Gaussian and is commonly used for edge detection or identifying transient features in signals. Like the Morlet, it takes a 1D signal and scales to produce a time-frequency representation.

#### **b. *logger\_util.py***

This utility script establishes a robust logging mechanism for the entire application.

```

1. class DualLogger:
2.     # ... (full code as provided) ...
3.     def write(self, message):
4.         # ... (full code as provided) ...
5.     def flush(self):
6.         # ... (full code as provided) ...
7.     def close(self):
8.         # ... (full code as provided) ...

```

The `DualLogger` class redirects `sys.stdout` and `sys.stderr` to simultaneously write all console output (for example, print statements, warnings, errors) to both the actual terminal and a specified log file (`debug_output.txt`). It includes `flush()` and `os.fsync()` to ensure messages are written to disk immediately, preventing data loss during unexpected application termination.

```

1. def handle_exception(exc_type, exc_value, exc_traceback):
2.     if isinstance(exc_type, KeyboardInterrupt):
3.         # Allow Ctrl+C to stop the program normally without logging it as a
         crash
4.         sys.__excepthook__(exc_type, exc_value, exc_traceback)
5.         return
6.
7.     # Format the error message header
8.     error_header = "\n" + "!"*60 + "\n"
9.     error_title = "        CRITICAL SYSTEM ERROR (UNHANDLED
        EXCEPTION)  \n"
10.    error_footer = "!"*60 + "\n"
11.
12.    # Get the string representation of the full traceback
13.    trace_str = "".join(traceback.format_exception(exc_type, exc_value,
        exc_traceback))
14.

```

```

15.     full_error_msg                                     =
        f'{error_header}{error_title}{error_footer}\n{trace_str}\n'
16.
17.     # Print to console (using the original stdout just in case our logger is
        broken)
18.     print(full_error_msg)
19.
20.     # Write to log file if the logger is active
21.     if _logger_instance and _logger_instance.log_file:
22.         _logger_instance.log_file.write(full_error_msg)
23.         _logger_instance.flush()

```

The `handle_exception` function is a global exception hook (`sys.excepthook`). If any unhandled Python exception occurs anywhere in the application (including GUI crashes), this function is automatically called. It formats the full traceback and writes it to the log file and console, providing critical debugging information that might otherwise be lost.

```

1.  def setup_logging(filename="debug_output.txt"):
2.      global _logger_instance
3.
4.      # Create the dual logger instance
5.      _logger_instance = DualLogger(filename)
6.
7.      # Redirect standard output (print statements)
8.      sys.stdout = _logger_instance
9.
10.     # Redirect standard error (tracebacks and warnings)
11.     sys.stderr = _logger_instance
12.
13.     # Hook into the global exception handler
14.     sys.excepthook = handle_exception
15.
16.     print("[SYSTEM] Debug Logger Initialized.")
17.     print(f"[SYSTEM] Log file location: {os.path.abspath(filename)}")
18.     print("[SYSTEM] Global Exception Hook is active.")

```

The `setup_logging` function initializes the `DualLogger` and installs the `handle_exception` hook. It is designed to be called once at the very beginning of the `main.py` script.

### c. *load\_data\_eeg\_mne.py*

This script handles the ingestion of GDF EEG data files and initial data structuring.

```

1.  def load_eeg_data(filepath):
2.      # 1. Validation
3.      if not os.path.exists(filepath):
4.          print(f"[ERROR] File not found: {filepath}")
5.          return None, None, None, None, None
6.

```

```

7.     filename = os.path.basename(filepath)
8.     print(f"[INFO] Loading File: {filename}")
9.
10.    # 2. Read Raw GDF (Preload required for filtering)
11.    try:
12.        raw = mne.io.read_raw_gdf(filepath, preload=True,
    verbose='ERROR')
13.    except Exception as e:
14.        print(f"[ERROR] MNE Load Failed for {filename}: {e}")
15.        return None, None, None, None, None
16.
17.    # 3. Rename Channels (Standardize BCI IV 2b naming)
18.    original_names = raw.ch_names
19.    rename_map = {}
20.
21.    # EEG 1-3 are C3, Cz, C4. EOG 4-6 are EOGs.
22.    if len(original_names) >= 3:
23.        rename_map[original_names[0]] = 'C3'
24.        rename_map[original_names[1]] = 'Cz'
25.        rename_map[original_names[2]] = 'C4'
26.
27.    if len(original_names) >= 6:
28.        rename_map[original_names[3]] = 'EOG:01'
29.        rename_map[original_names[4]] = 'EOG:02'
30.        rename_map[original_names[5]] = 'EOG:03'
31.
32.    raw.rename_channels(rename_map)
33.
34.    # 4. Set Montage (Standard 10-20 System)
35.    try:
36.        montage = mne.channels.make_standard_montage('standard_1020')
37.        raw.set_montage(montage, on_missing='ignore')
38.    except Exception as e:
39.        print(f"[WARN] Montage setting failed (Non-critical): {e}")
40.
41.    # 5. Extract Events with Comprehensive Mapping
42.    '768': 768, # Start of a trial
43.    '769': 769, # Cue onset LEFT (Class 1)
44.    '770': 770, # Cue onset RIGHT (Class 2)
45.    '781': 781, # BCI Feedback (Continuous)
46.    '783': 783, # Cue unknown
47.
48.    # Artifacts / States
49.    '276': 276, # Idling EEG (Eyes Open)
50.    '277': 277, # Idling EEG (Eyes Closed)
51.    '1023': 1023, # Rejected trial (Artifact)

```

```

52.     '32766': 32766, # Start of a new run
53.
54.     # Hexadecimal Variants (MNE sometimes reads annotations as hex
    strings)
55.     '0x0300': 768,
56.     '0x0301': 769,
57.     '0x0302': 770,
58.     '0x030D': 781,
59.     '0x03FF': 1023,
60.     '0x7FFE': 32766
61. }
62.
63. # Scan file annotations
64. annotations = raw.annotations
65. unique_desc = np.unique(annotations.description)
66.
67. # Build a specific mapping for this file
68. used_map = {}
69. for desc in unique_desc:
70.     desc_str = str(desc).strip()
71.
72.     # Check strict string match
73.     if desc_str in standard_bci_map:
74.         used_map[desc_str] = standard_bci_map[desc_str]
75.     else:
76.         # Try integer conversion fallback
77.         try:
78.             val = int(desc_str)
79.             if val in standard_bci_map.values():
80.                 used_map[desc_str] = val
81.         except ValueError:
82.             continue
83.
84. if not used_map:
85.     print(f"[CRITICAL WARNING] No standard BCI events found in
    {filename}.")
86.     events, event_id_map = mne.events_from_annotations(raw,
    verbose=False)
87. else:
88.     print(f"[INFO] Applied Event Map for {filename}: {used_map}")
89.     events, event_id_map = mne.events_from_annotations(raw,
    event_id=used_map, verbose=False)
90.
91. # 6. Determine Session Type (Screening vs Feedback)
92. has_feedback = 781 in event_id_map.values()

```

```

93.     session_type = "Smiley Feedback" if has_feedback else "Screening (No
        Feedback)"
94.
95.     # 7. Channel Validation
96.     available_channels = raw.ch_names
97.     motor_channels = ['C3', 'Cz', 'C4']
98.     missing_channels = [ch for ch in motor_channels if ch not in
        available_channels]
99.
100.    if missing_channels:
101.        print(f"[CRITICAL ERROR] Missing required motor channels in
            {filename}: {missing_channels}")
102.        # We return None to signal the GUI to abort loading this specific
            file
103.        return None, None, None, None, None
104.
105.    # 8. Build Metadata Dictionary
106.    duration_sec = raw.times[-1]
107.
108.    # Count specific trials
109.    count_left = np.sum(events[:, 2] == 769)
110.    count_right = np.sum(events[:, 2] == 770)
111.    count_artifact = np.sum(events[:, 2] == 1023)
112.
113.    session_info = {
114.        "filename": filename,
115.        "type": session_type,
116.        "has_feedback": has_feedback,
117.        "total_events": len(events),
118.        "count_left": int(count_left),
119.        "count_right": int(count_right),
120.        "count_artifact": int(count_artifact),
121.        "sampling_rate": raw.info['sfreq'],
122.        "duration_sec": duration_sec,
123.        "duration_str": f"{int(duration_sec // 60)} min {int(duration_sec %
            60)} sec",
124.        "all_channels": available_channels,
125.        "motor_channels": motor_channels
126.    }
127.
128.    print(f"[INFO] Loaded {filename} | Type: {session_type} | Trials:
        {count_left+count_right}")
129.
130.    return raw, events, event_id_map, raw.info['sfreq'], session_info

```

The `load_eeeg_data` function is responsible for loading a single .gdf file using the mne library. It performs several critical steps:

- Raw Data Loading: Reads the GDF file, preloading all data into memory.
- Channel Renaming: Standardizes channel names (for example, EEG 001 to C3, EOG 001 to EOG:01) based on the BCI Competition IV 2b dataset description.
- Montage Setting: Applies a standard 10-20 electrode montage for potential topographic visualization.
- Event Extraction & Mapping: Parses event annotations (for example, 769, 770, 783) from the GDF file and maps them to standard integer IDs. This step identifies Cue Onset Left/Right and Unknown Cue events.
- Session Type Detection: Infers whether the session is a "Screening (No Feedback)" or "Smiley Feedback" session by checking for the presence of the 781 (BCI Feedback) event.
- Motor Channel Extraction: Extracts only the C3, Cz, C4 channels from the raw data, converts their amplitude to microvolts (uV), and stores this as a 2D NumPy array.
- Metadata Generation: Compiles comprehensive session\_info (filename, type, duration, trial counts) for use by the GUI.

#### d. *CWT.py*

This script acts as the Python interface for the C++ CWT implementations, enabling time-frequency analysis.

```

1. def run_cwt_single(eeg_data, fs, f_min=4, f_max=40, f_step=1.0,
   wavelet_type='morlet'):
2.     """
3.     Computes CWT for a single 1D array. Helper function.
4.     """
5.     if lib is None:
6.         raise RuntimeError("C++ Library is not loaded.")
7.
8.     n_samples = len(eeg_data)
9.     freqs = np.arange(f_min, f_max + 0.1, f_step)
10.
11.    # Scale conversion (s = fs / f)
12.    with np.errstate(divide='ignore'):
13.        scales = fs / freqs
14.
15.    n_scales = len(scales)
16.
17.    # Prepare Ctypes Arrays
18.    input_c = np.ascontiguousarray(eeg_data, dtype=np.float64)
19.    scales_c = np.ascontiguousarray(scales, dtype=np.float64)
20.    output_c = np.zeros(n_scales * n_samples, dtype=np.float64)
21.
22.    # Call C++
23.    if wavelet_type == 'mexican_hat':
24.        lib.compute_cwt_mexican_hat(input_c, n_samples, scales_c, n_scales,
           fs, output_c)

```

```

25.     else:
26.         lib.compute_cwt_magnitude(input_c, n_samples, scales_c, n_scales,
            fs, output_c)
27.
28.     # Reshape (Freqs x Time)
29.     tfr_data = output_c.reshape((n_scales, n_samples))
30.
31.     return tfr_data, freqs

```

The `run_cwt_single` function is the core CWT runner. It takes a 1D EEG signal, converts a desired frequency range (`f_min`, `f_max`, `f_step`) into wavelet scales, prepares C-compatible NumPy arrays, and calls either the `compute_cwt_magnitude` (Morlet) or `compute_cwt_mexican_hat` (Mexican Hat) function from the `eeg_core.cpp` DLL. It returns a 2D time-frequency representation (`tfr_data`) and the `freqs` axis.

```

1. def run_cwt(eeg_data, fs, f_min=4, f_max=40, f_step=1.0,
    wavelet_type='morlet'):
2.     return run_cwt_single(eeg_data, fs, f_min, f_max, f_step, wavelet_type)

```

This function is a wrapper that simply calls `run_cwt_single`. It exists to maintain compatibility with earlier GUI calls expecting a `run_cwt` function that processes a single channel.

#### e. *filtering\_BPF\_EEG.py*

This module provides bandpass filtering functionality, critical for isolating target EEG rhythms.

```

1. def get_filter_description(low=0.5, high=30.0):
2.     # ... (full code as provided) ...

```

The `get_filter_description` function generates an educational text explaining the purpose and parameters of the Bandpass Filter. It covers why specific low and high cutoff frequencies (for example, 0.5Hz for drift, 30Hz for muscle artifacts) are chosen to preserve relevant rhythms.

```

3. def design_butter_bandpass_2nd_order(lowcut, highcut, fs):
4.     # Angular frequency
5.     w0 = 2 * np.pi * np.sqrt(lowcut * highcut) / fs
6.     bw_hz = highcut - lowcut
7.     Q = np.sqrt(lowcut * highcut) / bw_hz
8.
9.     # Intermediate variables for Bilinear Transform
10.    alpha = np.sin(w0) / (2 * Q)
11.    cos_w0 = np.cos(w0)
12.
13.    # Calculate coefficients
14.    b0 = alpha
15.    b1 = 0.0
16.    b2 = -alpha
17.
18.    a0 = 1 + alpha

```



```

19.  a1 = -2 * cos_w0
20.  a2 = 1 - alpha
21.
22.  # Normalize by a0 so that a[0] becomes 1 (Standard IIR format)
23.  b = np.array([b0, b1, b2]) / a0
24.  a = np.array([a0, a1, a2]) / a0
25.
26.  return b, a

```

The `design_butter_bandpass_2nd_order` function calculates the numerator (b) and denominator (a) coefficients for a 2nd-order Butterworth bandpass filter. It implements the filter design formulas using pure NumPy, adhering to the constraint of not using `scipy.signal.butter`.

```

1.  def run_filter_single(eeg_data, fs, low=0.5, high=30.0, order=2):
2.      if lib is None:
3.          raise RuntimeError("C++ Library not loaded. Cannot perform Filtering.")
4.
5.      if eeg_data.ndim != 1:
6.          eeg_data = eeg_data.flatten()
7.
8.      # 1. Calculate Coefficients (Pure Numpy)
9.      b, a = design_butter_bandpass_2nd_order(low, high, fs)
10.
11.     # Prepare Ctypes arrays for coefficients
12.     b_c = np.ascontiguousarray(b, dtype=np.float64)
13.     a_c = np.ascontiguousarray(a, dtype=np.float64)
14.
15.     # 2. Prepare Data Arrays
16.     temp_input = np.ascontiguousarray(eeg_data, dtype=np.float64)
17.     output = np.zeros_like(temp_input)
18.
19.     # 3. First Pass (2nd Order)
20.     lib.apply_filter(temp_input, len(temp_input),
21.                     b_c, len(b_c),
22.                     a_c, len(a_c),
23.                     output)
24.
25.     # 4. (Optional) Second Pass for Steepness (Pseudo-4th Order)
26.     # This cascades the filter to achieve a sharper cutoff (-24dB/octave)
27.     if order >= 4:
28.         # Swap input/output for the next pass
29.         # We take the output of the first pass as input for the second
30.         temp_input = output.copy()
31.         lib.apply_filter(temp_input, len(temp_input),
32.                         b_c, len(b_c),
33.                         a_c, len(a_c),
34.                         output)

```

```
35.  
36. return output
```

The `run_filter_single` function applies the designed Butterworth filter to a single 1D EEG channel. It handles converting Python NumPy arrays to C-compatible pointers and calls the `apply_filter` function from `eeg_core.cpp`. It also supports cascading the 2nd-order filter to create a sharper 4th-order response.

```
1. def run_filter_multi_channel(eeg_data_3ch, fs, low=0.5, high=30.0, order=2):  
2.     n_channels, n_samples = eeg_data_3ch.shape  
3.     filtered_data = np.zeros_like(eeg_data_3ch)  
4.  
5.     channel_names = ['C3', 'Cz', 'C4']  
6.  
7.     for i in range(n_channels):  
8.         # ch_name = channel_names[i] if i < 3 else f'Ch {i}'  
9.         # print(f'[INFO] Filtering Channel {ch_name} with {low}-{high} Hz...')  
10.        filtered_data[i, :] = run_filter_single(eeg_data_3ch[i, :], fs, low, high,  
11.                                                order)  
12.    return filtered_data
```

The `run_filter_multi_channel` function is a wrapper that takes a 2D array (3 channels: C3, Cz, C4) and iterates through each channel, applying `run_filter_single` to it. This ensures all three target channels are filtered.

#### ***f. squaring\_EEG.py***

This module converts filtered EEG amplitude into instantaneous power.

```
1. def get_squaring_description():  
2.     # ... (full code as provided) ...
```

The `get_squaring_description` function provides educational text explaining why squaring is performed. It covers the concepts of signal rectification (making all values positive) and energy quantification, which are critical for averaging ERD/ERS.

```
1. def square_signal(eeg_data):  
2.     # Using numpy's optimized element-wise power operation.  
3.     # This works automatically for 1D arrays or 2D matrices (3 channels).  
4.     power_data = np.square(eeg_data)  
5.  
6.     return power_data
```

The `square_signal` function calculates the square of every element in the input `eeg_data` NumPy array. It is efficient as `np.square` is highly optimized and works directly on multi-dimensional arrays, thus supporting multi-channel input (C3, Cz, C4).

#### ***g. average\_all\_EEG\_trials.py***

This script performs synchronous averaging to enhance the signal-to-noise ratio of event-related responses.

```
1. def get_averaging_description():  
2.     # ... (full code as provided) ...
```

The `get_averaging_description` function explains the principle of synchronous averaging, how it improves the Signal-to-Noise Ratio ( $\sqrt{N}$  improvement), and its importance for revealing subtle event-related brain activity.

```
1. def extract_and_average_epochs(squared_data, events, fs, tmin=-1.0,
2.    tmax=4.0):
3.     # 1. Define Event IDs (BCI Competition IV 2b standard)
4.     EVENT_LEFT = 769
5.     EVENT_RIGHT = 770
6.
7.     # 2. Calculate Sample Offsets
8.     offset_start = int(tmin * fs)
9.     offset_end = int(tmax * fs)
10.    epoch_len = offset_end - offset_start
11.
12.    # Containers for trials
13.    trials_left = []
14.    trials_right = []
15.
16.    n_channels, n_samples = squared_data.shape
17.
18.    # 3. Iterate through events
19.    for event in events:
20.        sample_idx = event[0]
21.        event_id = event[2]
22.
23.        # Check boundaries to avoid crashes at start/end of file
24.        start_idx = sample_idx + offset_start
25.        end_idx = sample_idx + offset_end
26.
27.        if start_idx < 0 or end_idx > n_samples:
28.            continue
29.
30.        # Extract the segment (Epoch)
31.        # Shape: (n_channels, epoch_len)
32.        epoch_data = squared_data[:, start_idx:end_idx]
33.
34.        # Sort by Class
35.        if event_id == EVENT_LEFT:
36.            trials_left.append(epoch_data)
37.        elif event_id == EVENT_RIGHT:
38.            trials_right.append(epoch_data)
39.
40.    # 4. Convert to Numpy Arrays and Average
41.    # Stack Shape: (n_trials, n_channels, n_time_points)
42.    # Mean Axis: 0 (Average across trials, preserving channels and time)
```

```

43. # Left Hand Class
44. if len(trials_left) > 0:
45.     stack_left = np.array(trials_left)
46.     avg_left = np.mean(stack_left, axis=0)
47.     print(f'[INFO] Averaged {len(trials_left)} Left Hand trials.')
48. else:
49.     avg_left = np.zeros((n_channels, epoch_len))
50.     print("[WARN] No Left Hand trials found.")
51.
52. # Right Hand Class
53. if len(trials_right) > 0:
54.     stack_right = np.array(trials_right)
55.     avg_right = np.mean(stack_right, axis=0)
56.     print(f'[INFO] Averaged {len(trials_right)} Right Hand trials.')
57. else:
58.     avg_right = np.zeros((n_channels, epoch_len))
59.     print("[WARN] No Right Hand trials found.")
60.
61. # Generate Time Axis for plotting
62. time_axis = np.linspace(tmin, tmax, epoch_len)
63.
64. return avg_left, avg_right, time_axis

```

The `extract_and_average_epochs` function segments the continuous `squared_data` into individual trials (epochs) based on event markers. It separates these epochs by class (Left Hand, Right Hand) and then computes the mean (average) for each class across all trials. This yields a smooth, averaged power curve for each channel (C3, Cz, C4) representing the event-related activity.

#### ***h. moving\_average\_EEG.py***

This script applies a smoothing filter to the averaged power signals.

```

1. def get_smoothing_description():
2.     # ... (full code as provided) ...

```

The `get_smoothing_description` function provides educational text describing the Moving Average (MAV) process. It explains MAV as a low-pass filter in the time domain, used to extract the underlying "envelope" or trend of the noisy power signal.

```

1. def apply_moving_average(power_data, fs, window_sec=0.5):
2.     if lib is None:
3.         raise RuntimeError("C++ Library not loaded. Cannot perform Smoothing.")
4.
5.     # Handle 1D input by reshaping to 2D (1, N) temporarily
6.     is_1d = False
7.     if power_data.ndim == 1:
8.         power_data = power_data.reshape(1, -1)
9.         is_1d = True
10.

```

```

11. n_channels, n_samples = power_data.shape
12. window_samples = int(window_sec * fs)
13.
14. # Initialize output array
15. smoothed_data = np.zeros_like(power_data, dtype=np.float64)
16.
17. # Process each channel individually
18. for i in range(n_channels):
19.     # 1. Prepare C-compatible arrays (Contiguous memory)
20.     channel_in = np.ascontiguousarray(power_data[i], dtype=np.float64)
21.     channel_out = np.zeros(n_samples, dtype=np.float64)
22.
23.     # 2. Call C++ Function
24.     lib.moving_average(channel_in, n_samples, window_samples,
25.                         channel_out)
26.     # 3. Store result
27.     smoothed_data[i, :] = channel_out
28.
29. # Restore original shape if input was 1D
30. if is_1d:
31.     return smoothed_data.flatten()
32.
33. return smoothed_data

```

The `apply_moving_average` function takes the averaged power data (potentially multi-channel) and applies the moving average filter. It calls the `moving_average` function from `eeg_core.cpp` for each channel iteratively, leveraging the C++ backend for performance.

#### *i. **percentage\_ERD\_ERS.py***

This script quantifies the power changes as a percentage relative to a baseline, defining ERD and ERS.

```

1. def get_erd_description():
2.     # ... (full code as provided) ...

```

The `get_erd_description` function explains the mathematical formula for ERD/ERS percentage, differentiating between Activity (A) and Reference (R) periods. It details the physiological interpretation of negative values (ERD/activation) and positive values (ERS/idling).

```

1. def calculate_erd_percent(smoothed_data, fs, tmin, tmax, ref_interval=(-1.0,
2.     0.0)):
3.     n_channels, n_samples = smoothed_data.shape
4.
5.     # 1. Generate Time Axis
6.     time_axis = np.linspace(tmin, tmax, n_samples)
7.
8.     # 2. Identify Baseline Indices

```

```

8.     t_ref_start, t_ref_end = ref_interval
9.
10.    # Using searchsorted to find nearest indices efficiently
11.    idx_start = np.searchsorted(time_axis, t_ref_start)
12.    idx_end = np.searchsorted(time_axis, t_ref_end)
13.
14.    # Safety Checks
15.    if idx_start >= idx_end:
16.        print("[WARN] Invalid reference interval indices. Defaulting to first 10
            samples.")
17.        idx_start = 0
18.        idx_end = 10
19.
20.    if idx_end > n_samples:
21.        print("[WARN] Reference interval ends outside data bounds. Clipping to
            end.")
22.        idx_end = n_samples
23.
24.    # 3. Calculate Reference Power (R)
25.    R = np.mean(smoothed_data[:, idx_start:idx_end], axis=1, keepdims=True)
26.
27.    # Prevent Division by Zero
28.    # If R is exactly zero, set to a tiny epsilon to avoid NaN/Inf errors.
29.    R[R == 0] = 1e-9
30.
31.    # 4. Apply Formula: ((A - R) / R) * 100
32.    erd_percentage = ((smoothed_data - R) / R) * 100.0
33.
34.    return erd_percentage, time_axis

```

The `calculate_erd_percent` function computes the percentage change in power. It identifies a `ref_interval` (baseline period, typically pre-cue), calculates the average power ( $R$ ) during this baseline, and then applies the formula  $ERD\% = \frac{A-R}{R} \times 100$  to the entire `smoothed_data` time course. This process supports multi-channel input (C3, Cz, C4) using NumPy's broadcasting capabilities.

**j. *csp\_scratch.py***

This script implements Common Spatial Pattern (CSP) and additional temporal feature extraction.

```

1. def get_csp_description():
2.     # ... (full code as provided) ...

```

The `get_csp_description` function provides a detailed explanation of CSP, its objective to maximize variance separation between two classes, and the mathematical steps involved (covariance matrices, eigenvalue decomposition, whitening). It also describes the temporal features extracted.

```

1. class CSP_Scratch:

```

```

2.     # ... (full code as provided) ...
3.     def fit(self, X, y):
4.         # ... (full code as provided) ...
5.     def transform(self, X):
6.         # ... (full code as provided) ...
7.     def _compute_avg_covariance(self, X_epochs):
8.         # ... (full code as provided) ...
9.     def plot_feature_scatter(self, features, labels, class_names=['Left', 'Right']):
10.        # ... (full code as provided) ...

```

The CSP\_Scratch class implements the CSP algorithm from scratch. The fit method learns the spatial filters by solving a generalized eigenvalue problem on the covariance matrices of the two classes. The transform method applies these filters to new data and extracts **Log-Variance** features. The plot\_feature\_scatter method visualizes these 2D features to show class separability.

```

1. class TemporalFeatureExtractor:
2.     # ... (full code as provided) ...
3.     def transform(self, X):
4.         # ... (full code as provided) ...
5.     def plot_feature_boxplot_all_metrics(self, features, labels, channel_idx=0,
6.                                         class_names=['Left', 'Right'], ch_name="C3"):
7.         # ... (full code as provided) ...

```

The TemporalFeatureExtractor class calculates **5 time-domain features** (Mean, Variance, StdDev, Skewness, Kurtosis) for each channel of the EEG epochs. The transform method returns these features in a flattened vector. The plot\_feature\_boxplot\_all\_metrics method visualizes the distribution of these metrics using boxplots, aiding in feature analysis.

#### k. *ml\_analysis.py*

This script manages the entire Machine Learning pipeline, including feature fusion, model training, evaluation, and inference.

```

1. def get_ml_description():
2.     # ... (full code as provided) ...

```

The get\_ml\_description function provides descriptive text explaining the ML pipeline, including the dual-band feature extraction strategy, the 8 models compared, and the metrics used for evaluation.

```

1. class ML_Pipeline:
2.     # ... (full code as provided) ...
3.     def _internal_bandpass_filter(self, data, fs, lowcut=8.0, highcut=30.0,
4.                                   order=5):
5.         # ... (full code as provided) ...
6.     def prepare_data(self, raw_data, events, fs, tmin=0.5, tmax=3.5,
7.                      mode='train'):
8.         # ... (full code as provided) ...
9.     def extract_combined_features(self, epochs, labels=None, is_training=True):
10.        # ... (full code as provided) ...

```

```

9.     def run_full_comparison(self, X_train_raw, y_train_raw, X_test_raw,
10.        y_test_raw):
11.         # ... (full code as provided) ...
12.     def predict_new_data(self, epochs_raw, model_name, fs=250.0):
13.         # ... (full code as provided) ...
14.     def get_prediction(self, model_name):
15.         # ... (full code as provided) ...
16.     def generate_learning_curve(self, model_name):
17.         # ... (full code as provided) ...
18.     def generate_loss_curve(self, model_name):
19.         # ... (full code as provided) ...
20.     def generate_confusion_matrix(self, model_name):
21.         # ... (full code as provided) ...
22.     def get_detailed_predictions(self, model_name):
23.         # ... (full code as provided) ...

```

The ML\_Pipeline class orchestrates the machine learning process:

- (i) `_internal_bandpass_filter`: A helper that applies filtering specific to ML feature extraction (8-30 Hz Mu/Beta bands).
- (ii) `prepare_data`: Segments raw data into epochs, handles different event types (train vs inference modes), and applies internal bandpass filtering.
- (iii) `extract_combined_features`: This is the core feature fusion method, calling `csp_scratch.CSP_Scratch` and `csp_scratch.TemporalFeatureExtractor` to get both spatial and temporal features, which are then concatenated.
- (iv) `run_full_comparison`: This is the main training method for the GUI. It accepts explicit `X_train_raw`, `y_train_raw` (from aggregated training files) and `X_test_raw`, `y_test_raw` (from the designated test file). It extracts features, standardizes them using `StandardScaler`, and then trains and evaluates 8 different machine learning models, storing their detailed metrics.
- (v) `predict_new_data`: This method handles inference on new, unlabeled epochs\_raw (for example, from an 'E' file). It reuses the learned CSP filters and scaler to predict class labels.
- (vi) Visualization Helpers: `get_prediction`, `generate_learning_curve`, `generate_loss_curve`, `generate_confusion_matrix`, and `get_detailed_predictions` generate the necessary data and Matplotlib figures/tables for display in the GUI's ML tab.

## 1. *main.py*

This is the application's entry point.

```

1. def system_preflight_check():
2.     # ... (full code as provided) ...

```

The `system_preflight_check` function verifies the presence of the compiled C++ `eeg_processing.dll` to ensure all core components are available before starting the application.

```

1. def main():
2.     # ... (full code as provided) ...

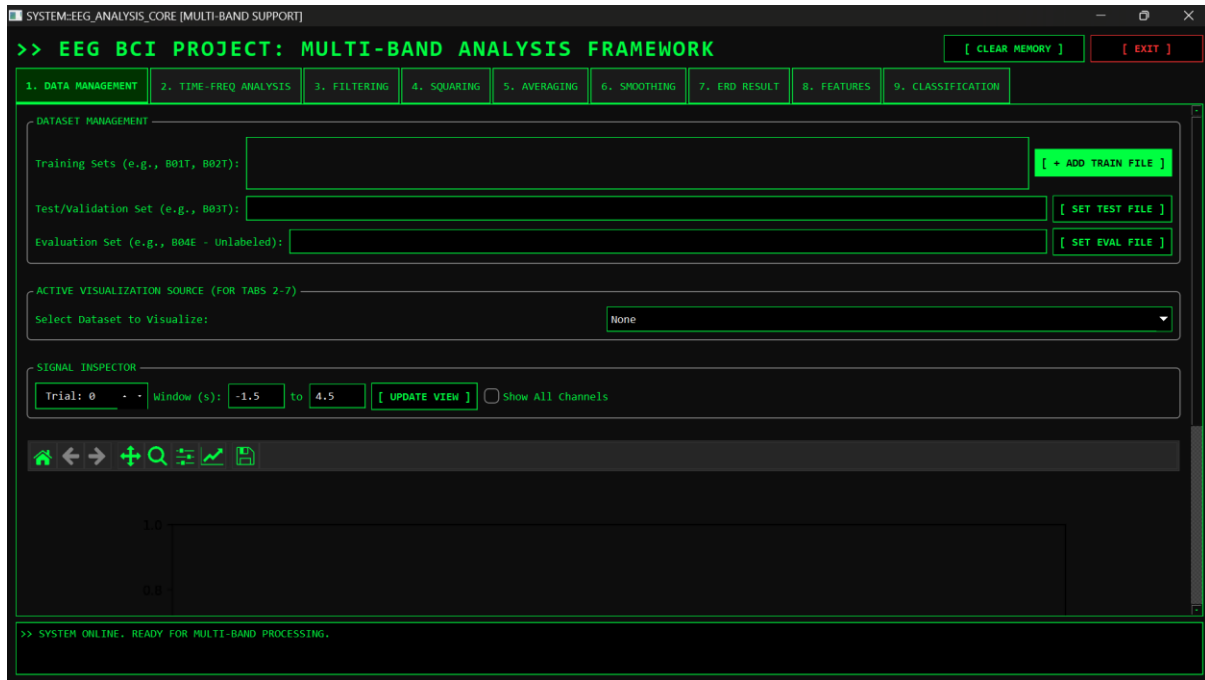
```



The main function initializes the logging system, performs pre-flight checks, loads necessary Python libraries, applies a global dark theme, and finally launches the EEGAnalysisWindow (GUI). It acts as the central orchestrator for the entire program startup.

## 2.3. Result of the Program and Analysis

### \*Initial GUI



*Figure 2.1. Initial GUI Components for the Program*

This program utilizes a comprehensive Graphical User Interface (GUI) designed to function as an end-to-end framework for BCI Motor Imagery analysis. As showed in **Figure 2.1**, the interface adopts a high-contrast "dark mode" aesthetic (green text on a black background), which is often favored in engineering applications for visual clarity during data inspection. The GUI is structured using a tabbed navigation system, guiding the user logically through the BCI pipeline, starting from Load Data tab and progressing through CWT, Filtering, Squaring, Averaging, Smoothing, ERD Result, Features, and finally Classification.

### 2.3.1. Dataset Choices and Explanation

The data used in this section is described in the details below:

#### For Training (CSP and ML):

B0101T.gdf

B0103T.gdf

B0201T.gdf

B0203T.gdf

B0302T.gdf

B0401T.gdf

B0403T.gdf

B0501T.gdf

B0601T.gdf

B0702T.gdf

B0801T.gdf

B0901T.gdf

B0902T.gdf

B0903T.gdf

**For Test 1:**

B0402T.gdf

**For Test 2:**

B0803T.gdf

**For Evaluation:**

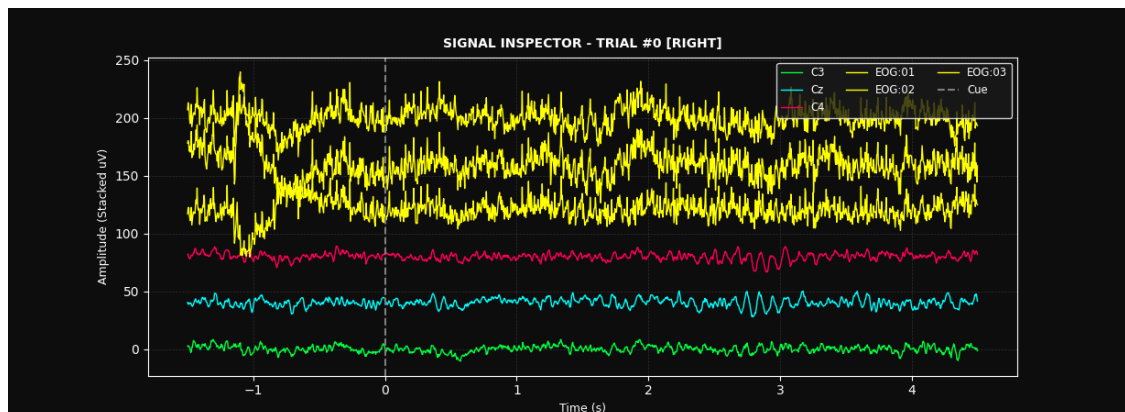
B0104E.gdf

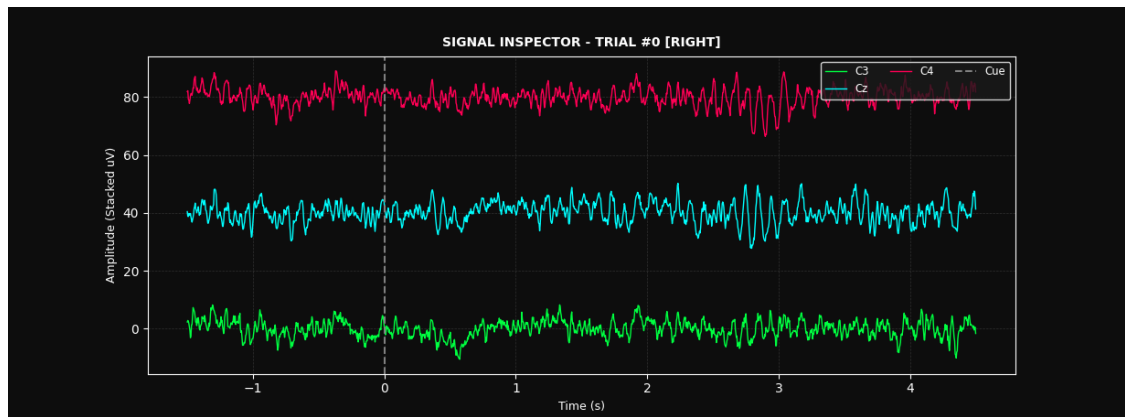
*Notes: For the Mu and Beta Band Program Result, it used the B0101T.gdf dataset, specifically the first trial.*

### 2.3.2. Program Result for the BPF using Mu/ $\mu$ Band (8-13 Hz)

#### 1. Load Data Tab

The process begins with the ingestion of the raw EEG signal, as visualized in **Figure 2.2**. This step utilizes the `load_data_eeg_mne.py` script described in Chapter II to parse the GDF files and extract the motor channels (C3, Cz, C4). Consistent with the theory in **Section 1.3**, the raw signal displayed here is non-stationary and visually noisy, containing a mix of all brain frequencies and potential artifacts. The figure specifically shows the stacked amplitude of these three channels for "Trial 1" of the *B0101T.gdf* dataset, serving as the raw input for the subsequent processing stages.

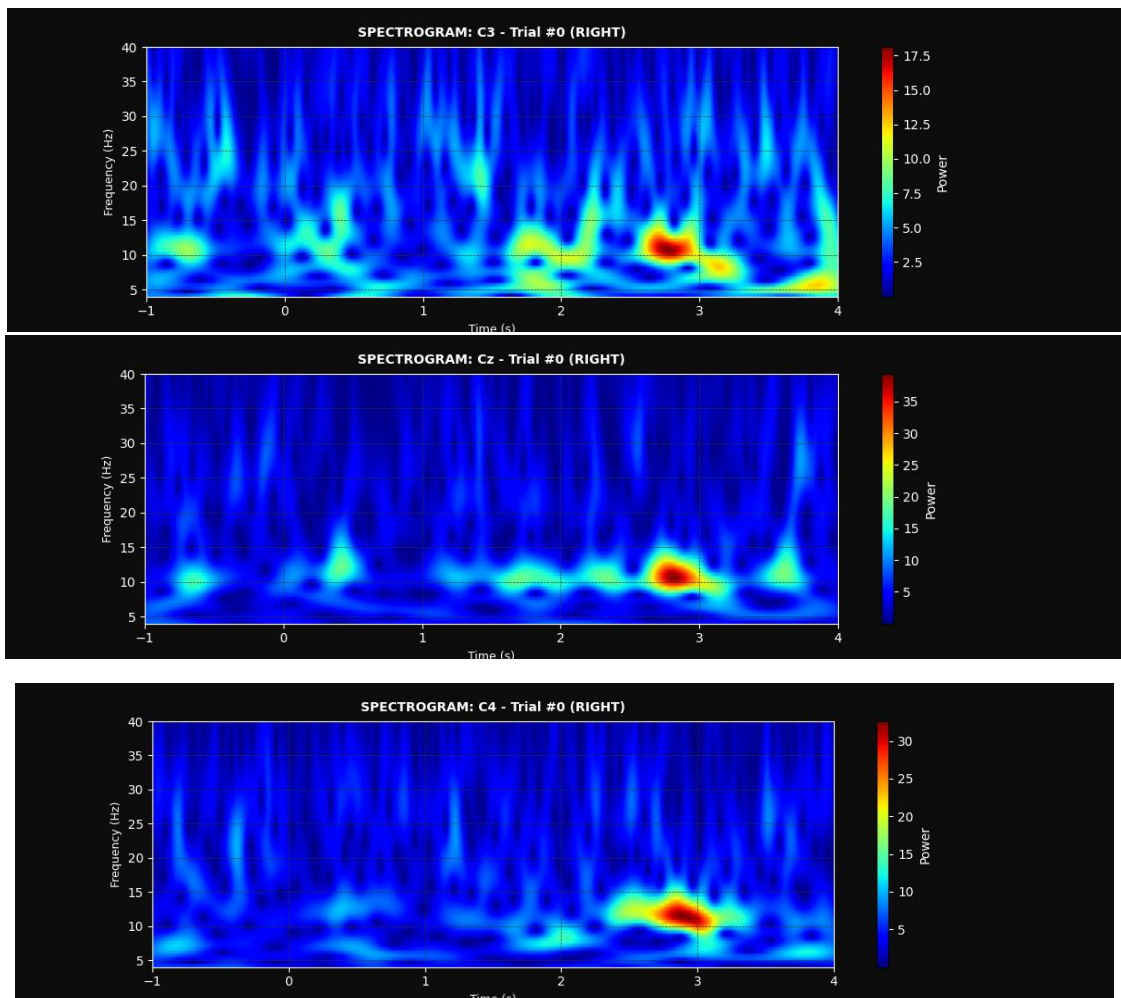




*Figure 2.2. The Graphs Result of the Load Data Tab for Trial 1 of Data B0101T.gdf*

## 2. CWT Tab

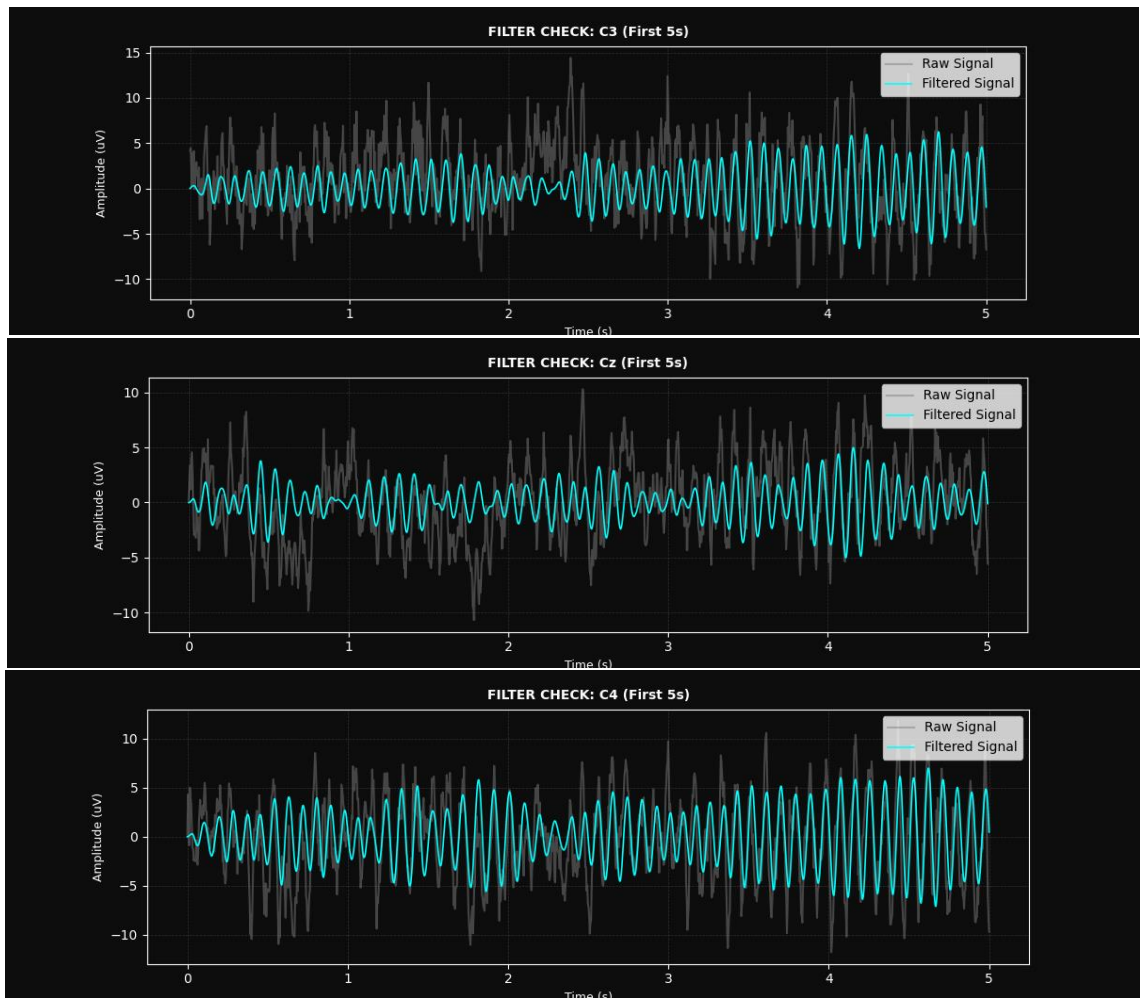
Before filtering, the program performs a Time-Frequency Analysis using the Continuous Wavelet Transform (CWT), implemented via CWT.py and the C++ backend. **Figure 2.3** presents the spectrograms for C3, Cz, and C4. This visualization aligns with **Section 1.3** of the theory, which states that CWT is essential for determining the precise temporal localization of brain rhythms. In the figure, we can observe the spectral energy distribution over time, providing a visual confirmation of where the activity (or lack thereof) occurs within the frequency domain before we isolate the specific bands.



**Figure 2.3.** The CWT Result on each Channel C3, Cz, and C4 for *Trial 1* of Data *B0101T.gdf*

### 3. BPF Filtering Tab

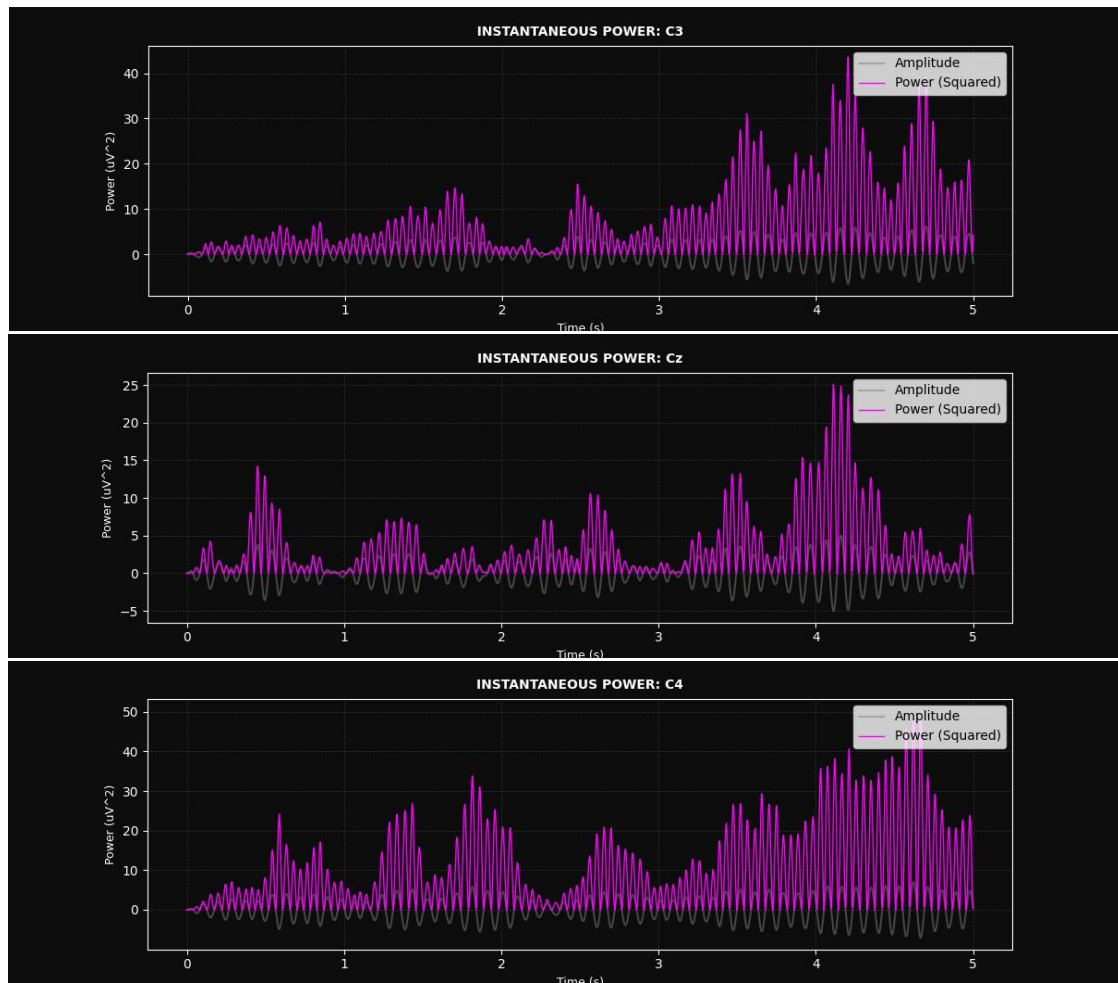
**Figure 2.4** illustrates the result of applying the Bandpass Filter (BPF), a critical preprocessing step detailed in **Section 1.3**. By utilizing the `filtering_BPF_EEG.py` module (which implements the Butterworth filter described in Chapter II), the software isolates the Mu Band (8–13 Hz). The figure clearly shows that the chaotic raw signal has been transformed into a clean, rhythmic oscillation. This step removes high-frequency muscle noise (EMG) and low-frequency drift, leaving only the specific neural oscillations relevant to the motor imagery task.



**Figure 2.4.** The 4<sup>th</sup> Order BPF Filtering Result focused in **Mu Band** on each Channel C3, Cz, C4 for *Trial 1* of Data *B0101T.gdf*

### 4. Squaring Tab

Following the theory of energy quantification, the filtered signals are squared to convert amplitude into power. **Figure 2.5** shows the output of the `squaring_EEG.py` script. As expected, the oscillating sine-like waves from the previous step are rectified (all values become positive), representing the *Instantaneous Power* of the signal. This step is a mathematical prerequisite for the subsequent averaging process, as averaging raw oscillatory signals would result in cancellation (summing to zero).



**Figure 2.5.** The Squaring Result focused on **Mu Band** on each Channel C3, Cz, C4 for **Trial 1 of Data B0101T.gdf**

## 5. Averaging Tab

To enhance the Signal-to-Noise Ratio (SNR) as discussed in the code explanation for `average_all_EEG_trials.py`, the program computes the Grand Average across all trials. **Figure 2.7** (using parameters from Figure 2.6) displays the distinct power trends for "Left Hand" (Cyan) versus "Right Hand" (Red) classes. This visualization validates the ERD/ERS theory (**Section 1.2**), where the difference in power between the two cognitive tasks begins to emerge, distinguishable from the random noise of single trials.

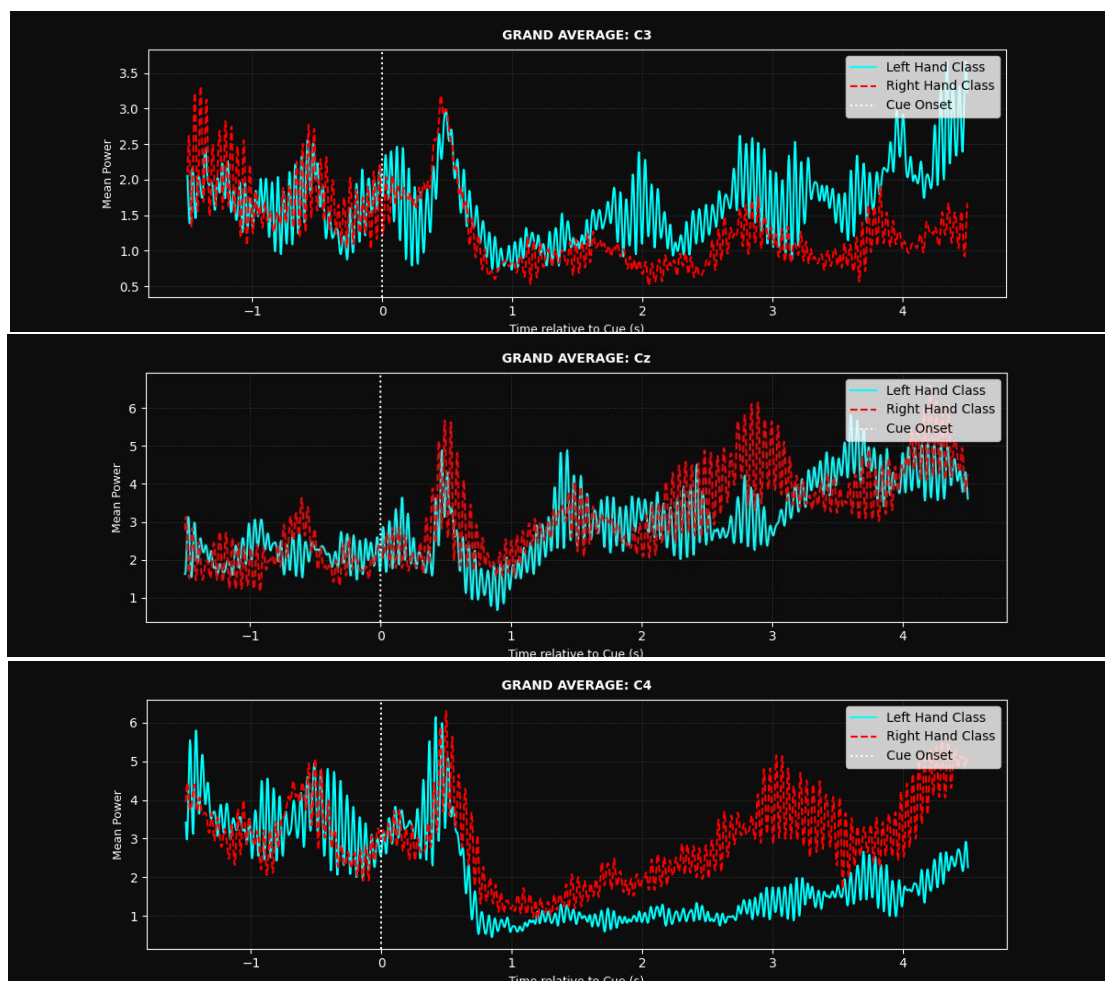
EPOCH PARAMETERS

T\_start [s]:

T\_end [s]:

**Figure 2.6.** The Input Parameters in Averaging Tab

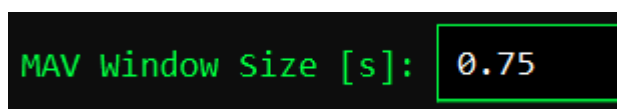




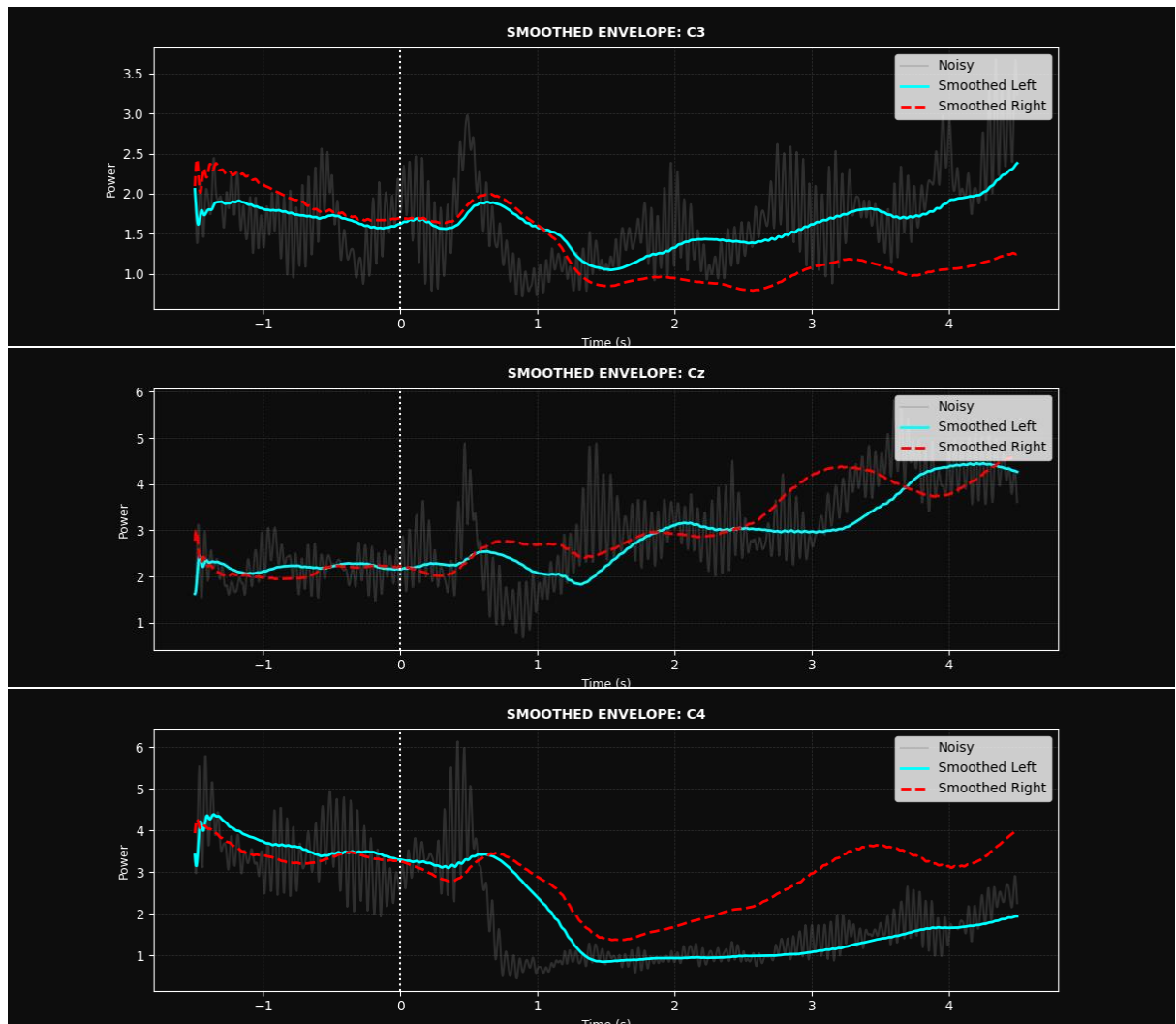
**Figure 2.7.** The Averaging Result focused on Mu Band on each Channel C3, Cz, C4 of Data B0101T.gdf

## 6. MAV (Smoothing) Tab

**Figure 2.9** presents the "Smoothed Envelope" of the averaged signals, achieved by applying a Moving Average Filter (window size 0.75s, as shown in **Figure 2.8**). This implementation, handled by `moving_average_EEG.py`, acts as a low-pass filter in the time domain. It effectively removes the high-frequency ripples from the squared power, resulting in a smooth curve that clearly depicts the trend of brain activity—specifically the power drop (desynchronization) or increase (synchronization) relative to the cue.



**Figure 2.8.** The MAV Window Size Parameter



**Figure 2.9.** The MAV Result focused on **Mu Band** on each Channel C3, Cz, C4 of Data **B0101T.gdf**

## 7. ERD/ERS (ERP) Percentage Tab

The final analytical step quantifies these changes using the ERD% formula described in **Section 1.2**:

$$((A - R)/R) \times 100.$$

**Figure 2.11** plots these percentages over time. A crucial observation is found in **Figure 2.12**, which presents the numerical average of the ERD% during the task window (0s–3s). The results show significant negative values (Left: -18.52%, Right: -40.36% on C3), which mathematically confirms the occurrence of Event-Related Desynchronization (ERD). This signifies that the motor cortex was activated during the motor imagery task, successfully validating the theoretical premise that movement imagination leads to a power drop in the Mu band.

BASELINE REFERENCE

Start [s]: 
End [s]:

**Figure 2.10.** The Parameters Input in ERD/ERS Percentage Tab



**Figure 2.11.** The MAV Result focused on **Mu Band** on each Channel C3, Cz, C4 of Data **B0101T.gdf**

```

=== AVERAGE ERD% DURING TASK (0s - 3s) ===
C3: Left=-18.52% | Right=-40.36%
Cz: Left=17.75% | Right=33.89%
C4: Left=-53.19% | Right=-32.12%

```

**Figure 2.12.** The Numerical Result for the ERD Percentage Tab on **Mu Band** on each Channel C3, Cz, C4 of Data **B0101T.gdf**

### 2.3.3. Program Result for the BPF using Beta/ $\beta$ Band (13-30 Hz)

#### 1. Load Data Tab

Same as before (for trial data test B0101T.gdf), because the data output for Load Data Tab is the same.

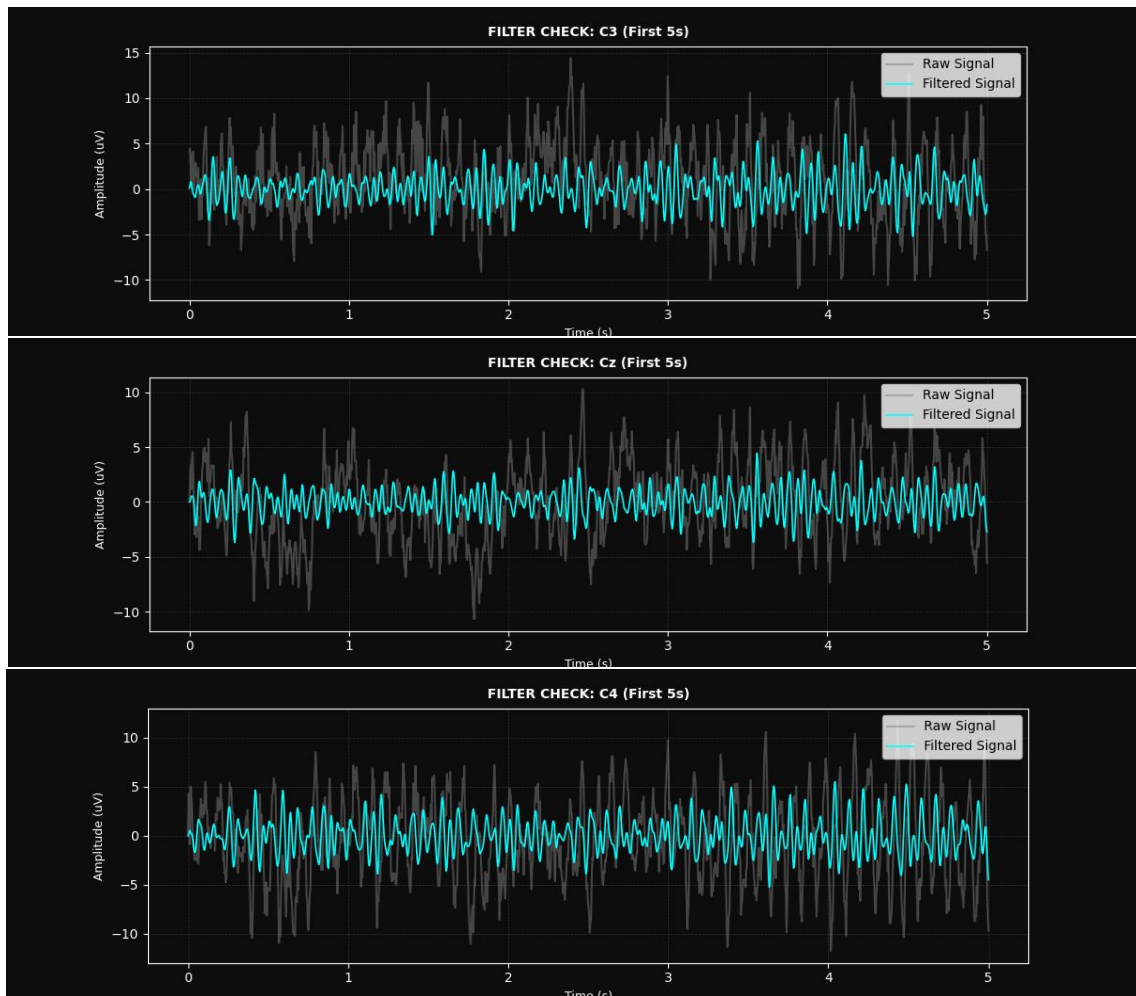
#### 2. CWT Tab

Same as before (for trial 1 in data test B0101T.gdf), because the data output for CWT Tab is the same.



### 3. BPF Filtering Tab

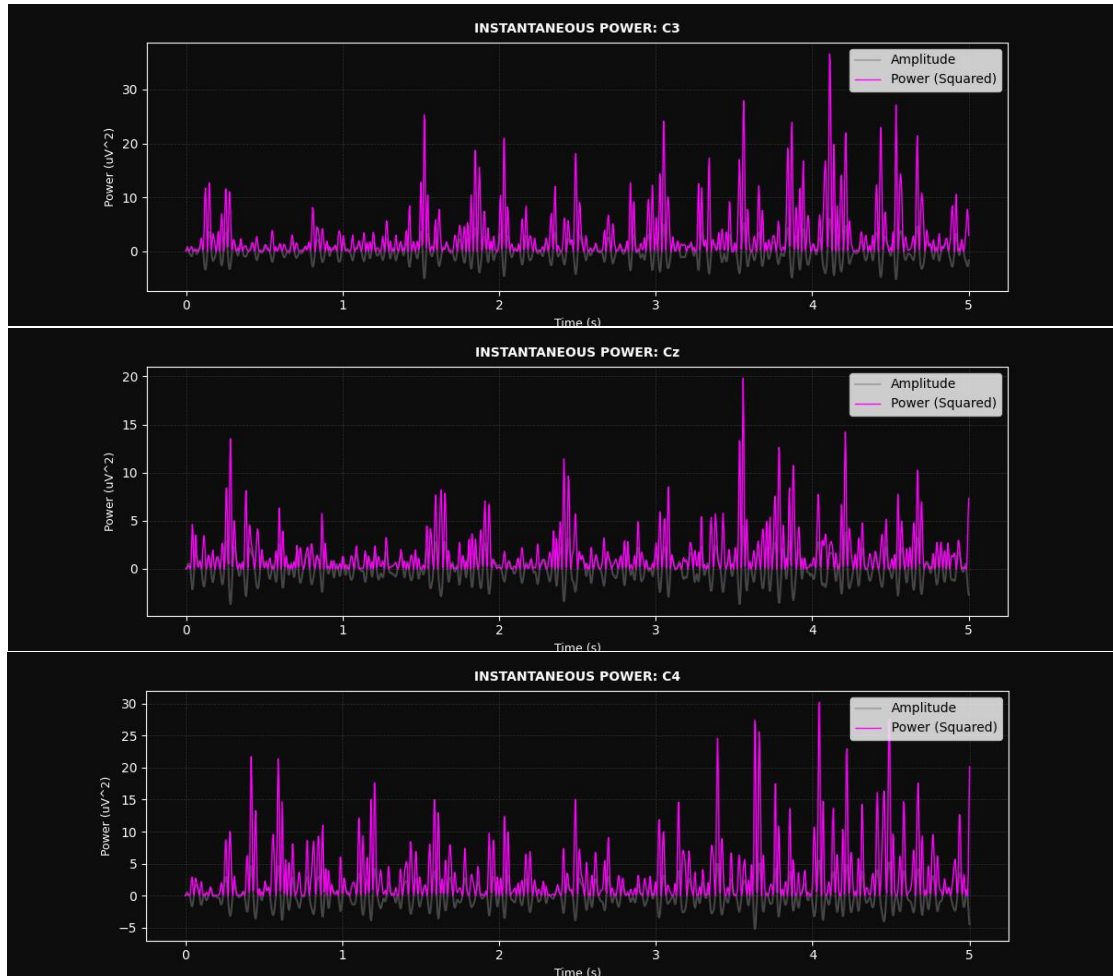
**Figure 2.13** marks the divergence point in the pipeline, where the Bandpass Filter is applied specifically to the **Beta Band (13–30 Hz)**. Utilizing the `filtering_BPF_EEG.py` module described in Chapter II, the Infinite Impulse Response (IIR) filter is adjusted to pass frequencies between 13 Hz and 30 Hz. Referring back to **Section 1.1** in Chapter I, the Beta band is identified as a critical sensorimotor rhythm associated with active concentration and motor tasks. The figure displays oscillations that are visibly faster and denser than the Mu band results, confirming the successful isolation of these higher-frequency components while suppressing the lower Mu waves and artifacts.



**Figure 2.13.** The 4<sup>th</sup> Order BPF Filtering Result focused in **Beta Band** on each Channel C3, Cz, C4 for **Trial 1 of Data B0101T.gdf**

### 4. Squaring Tab

Following the isolation of the Beta rhythm, the signal undergoes rectification, as shown in **Figure 2.14**. This process utilizes the `squaring_EEG.py` script to calculate the Instantaneous Power of the Beta band. As established in **Section 1.3**, this step is essential for converting the oscillatory amplitude into a positive energy metric. The figure demonstrates that the high-frequency Beta oscillations have been converted into a positive-only signal, effectively preparing the data for the synchronous averaging process that follows.

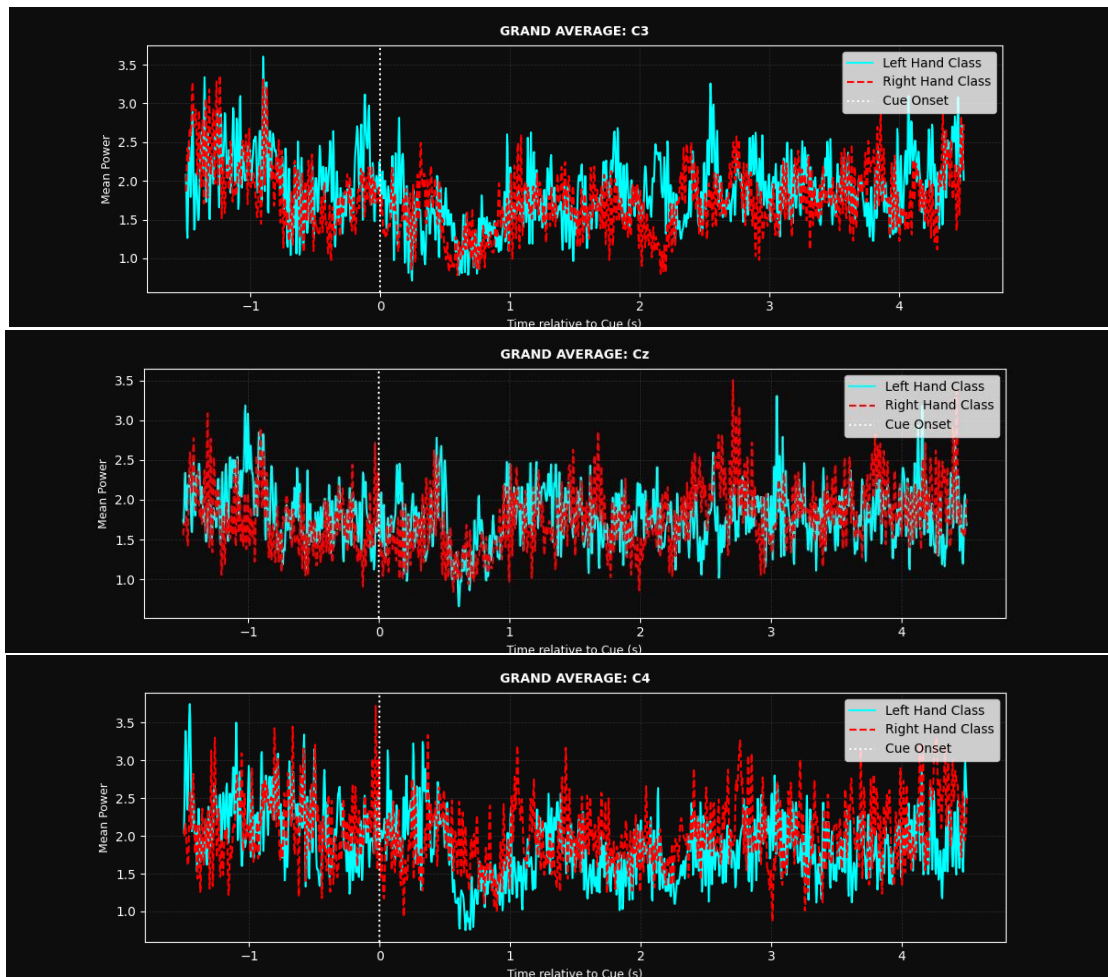


**Figure 2.14.** The Squaring Result focused on **Beta Band** on each Channel C3, Cz, C4 for Trial 1 of Data B0101T.gdf

## 5. Averaging Tab

The input parameters on this averaging tab is the same as in the Mu Band program result before.

**Figure 2.15** illustrates the Grand Average of the Beta power across all trials, separated by class (Left Hand vs. Right Hand). By employing the algorithms in `average_all_EEG_trials.py`, the system enhances the Signal-to-Noise Ratio (SNR). This visualization aligns with the ERD/ERS theory in **Section 1.2**, revealing how the Beta power fluctuates in response to the motor imagery cue. Unlike the single-trial data, this averaged view reveals distinct patterns of activation that are specific to the Beta range, which often shows sharper desynchronization dynamics compared to the Mu band.

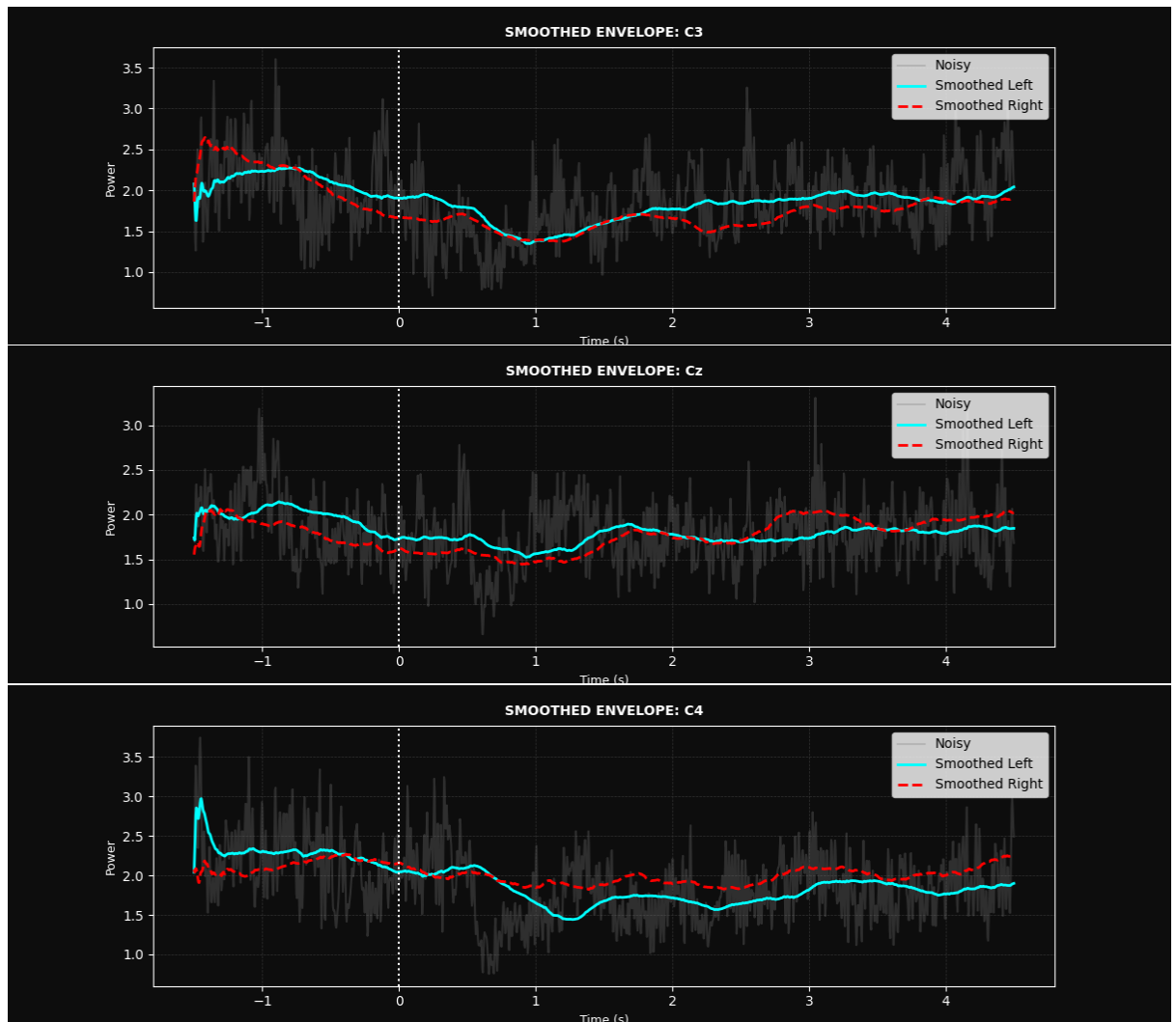


**Figure 2.15.** The Averaging Result focused on **Beta Band** on each Channel C3, Cz, C4 of Data B0101T.gdf

## 6. MAV (Smoothing) Tab

The input window size on this MAV tab is the same as in the Mu Band program result before.

To extract the envelope of the Beta activity, a Moving Average filter is applied, resulting in the smooth curves shown in **Figure 2.16**. Using the `moving_average_EEG.py` module, the high-frequency variance of the squared Beta signal is smoothed out. This visualization clearly depicts the temporal evolution of the brain's energy, allowing for a clearer visual comparison between the baseline state and the active motor imagery state. It highlights the sustained drop in power (ERD) that typically occurs during the imagination of movement.

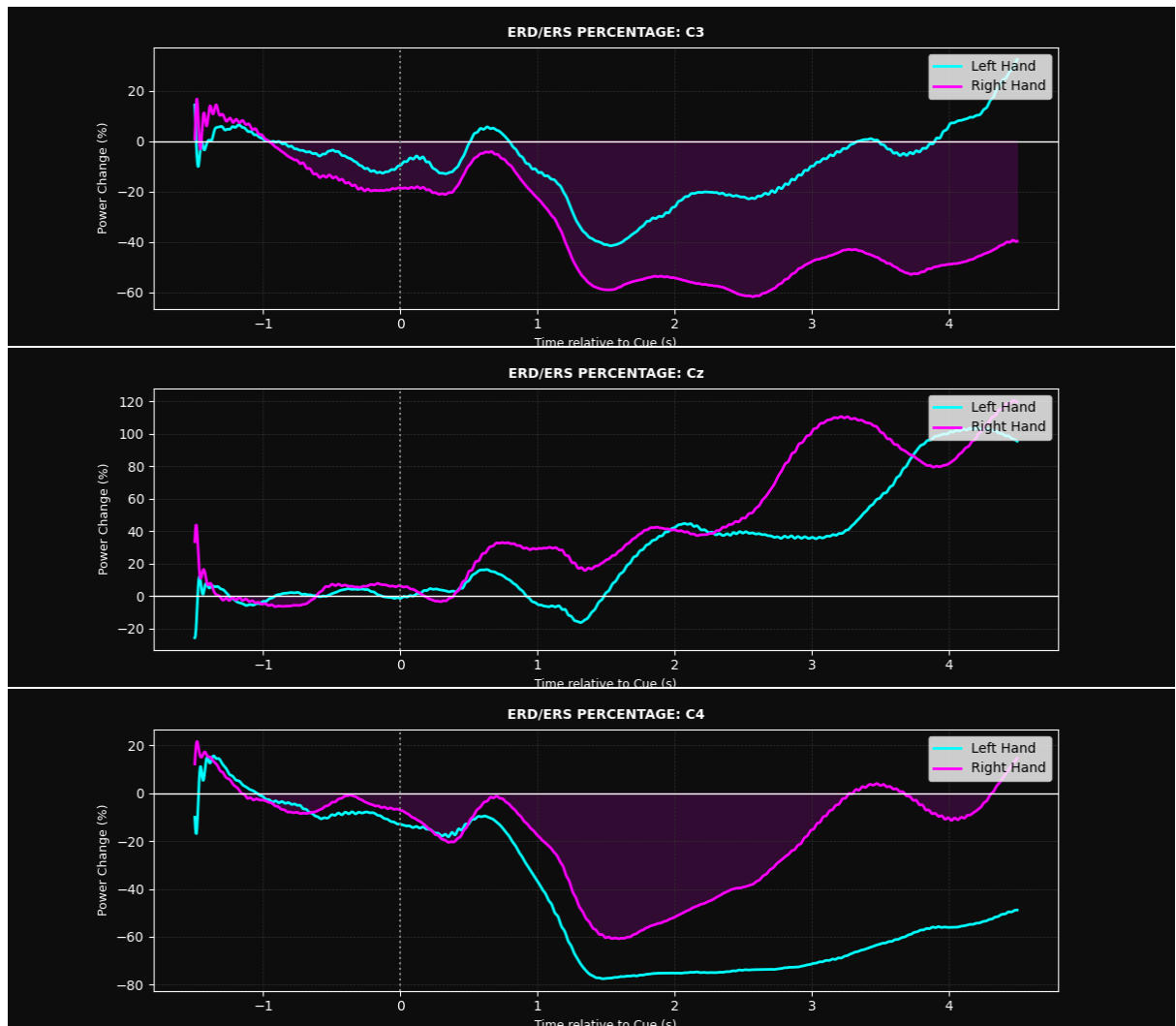


**Figure 2.16.** The MAV Result focused on **Beta Band** on each Channel C3, Cz, C4 of Data B0101T.gdf

## 7. ERD/ERS (ERP) Percentage Tab

The input parameters on this ERD/ERS Percentage tab is the same as in the Mu Band program result before.

The final analysis for the Beta band is the quantification of Event-Related Desynchronization (ERD), calculated using the formula from **Section 1.2**  $((A - R)/R \times 100)$ . **Figure 2.17** plots these percentage curves, while **Figure 2.18** provides the specific numerical results. The data shows distinct negative values (Left: -20.97%, Right: -31.94% on C3), which confirms that significant desynchronization occurs in the Beta band during motor imagery. This validates the premise in **Section 1.1** that Beta activity is a reliable indicator of motor cortex activation and serves as a complementary feature to the Mu band for robust classification.



**Figure 2.17.** The MAV Result focused on **Beta Band** on each Channel C3, Cz, C4 for of Data B0101T.gdf

```

=== AVERAGE ERD% DURING TASK (0s - 3s) ===
C3: Left=-20.97% | Right=-31.94%
Cz: Left=-15.78% | Right=-12.16%
C4: Left=-24.95% | Right=-7.43%

```

**Figure 2.18.** The Numerical Result for the ERD Percentage Tab on **Mu Band** on each Channel C3, Cz, C4 of Data B0101T.gdf

### ○ Detailed Analysis of Mu and Beta Band Results and Channel Implications

The comparative analysis of the Mu (8–13 Hz) and Beta (13–30 Hz) bands reveals critical insights into the neurophysiological dynamics of Motor Imagery and significantly influences the program's feature extraction strategy. In the Mu band, the results (**Figure 2.12**) demonstrate a classic pattern of hemispheric lateralization. Specifically, the C3 channel (Left Hemisphere) exhibits a stronger power drop during Right Hand trials (-40.36%) compared to Left Hand trials (-18.52%), while the C4 channel (Right Hemisphere) shows the inverse, with a deeper desynchronization for Left Hand trials (-53.19%) compared to Right Hand trials (-32.12%). This validates the fundamental theory

of contralateral motor control described in Chapter I. However, the Beta band results (**Figure 2.18**) show a more generalized desynchronization across all channels, with negative ERD values recorded for both C3, Cz, and C4 during both tasks. This suggests that while Mu rhythms are highly spatially specific to the idle/active state of the motor cortex, Beta rhythms represent a broader state of cognitive motor planning and active concentration. Consequently, the program utilizes this distinction by treating these bands as separate feature sets; the high spatial specificity of the Mu band drives the discrimination of "Left vs. Right," while the sustained ERD in the Beta band confirms the general presence of a "Motor Task," acting as a robust confirmation signal for the classifier.

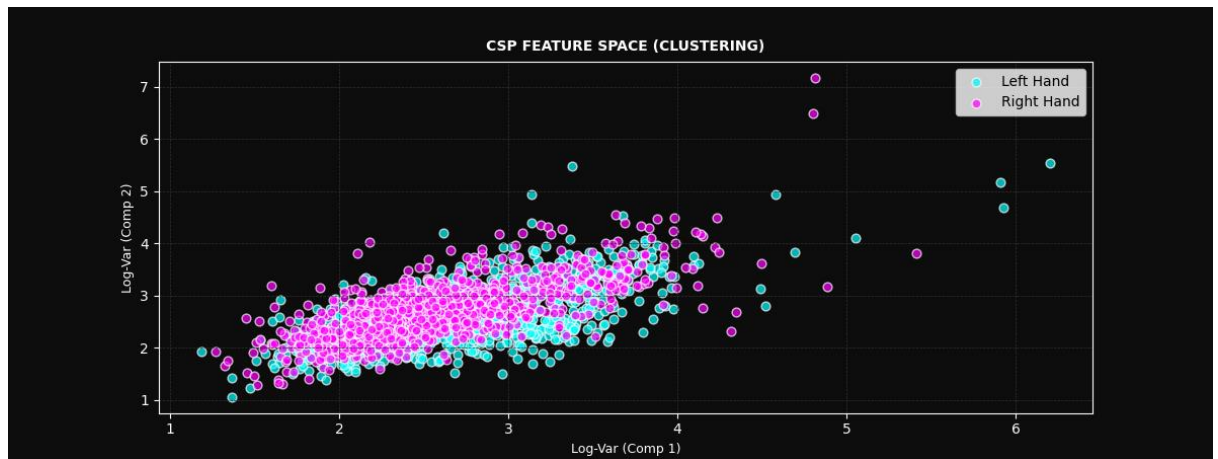
The behavior of the Cz channel (Vertex) presents the most distinct contrast between the two frequency bands, highlighting its role as a reference for spatial filtering. In the Mu band, Cz exhibits positive Event-Related Synchronization (ERS) values (+17.75% and +33.89%), indicating that the central vertex region remains in an idling or inhibitory state while the lateral motor areas are active. Conversely, in the Beta band, Cz shifts to a negative ERD state (-15.78% and -12.16%). This phenomenon implies that while the resting rhythm (Mu) remains synchronized at the center of the head, the cognitive processing rhythm (Beta) engages the entire sensorimotor strip. For the program's Common Spatial Pattern (CSP) algorithm, this contrast is vital. The divergence between the lateral channels (C3/C4) and the central channel (Cz) in the Mu band allows the CSP filters to maximize variance (discriminability) between classes, whereas the generalized activation in the Beta band adds a layer of complexity that helps the Machine Learning model distinguish true motor imagery from resting artifacts. Thus, utilizing all three channels across both bands is not merely redundant but essential for capturing the full spatiotemporal dynamics of the brain signal.

### **2.3.4. Program Result for the CSP and ML Tab for Data Test B0402T.gdf**

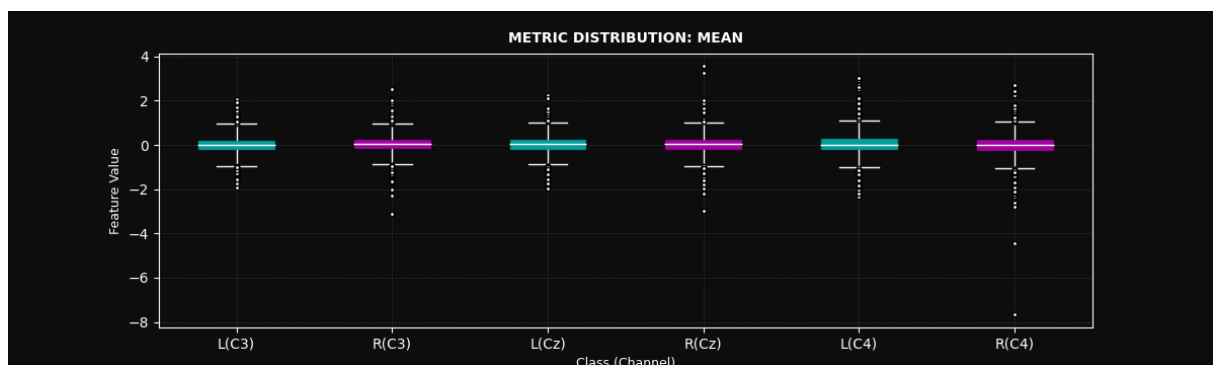
#### **1. CSP Tab Result**

The feature extraction process, driven by the `csp_scratch.py` module described in Chapter II, yields highly distinct patterns for this dataset. **Figure 2.19** visualizes the Common Spatial Pattern (CSP) feature space, where the "Left Hand" (Cyan) and "Right Hand" (Magenta) trials form two clearly separable clusters. This aligns with the theory in **Section 1.4**, which states that CSP maximizes the variance for one class while minimizing it for the other. Furthermore, **Figures 2.20** through **2.24** present the boxplot distributions for the temporal statistical features (Mean, Variance, Standard Deviation, Skewness, and Kurtosis). These figures demonstrate that beyond spatial variance, the statistical properties of the signal in the time domain also differ significantly between classes, particularly in the variance and standard deviation metrics, providing a robust multi-dimensional feature vector for the classifier.

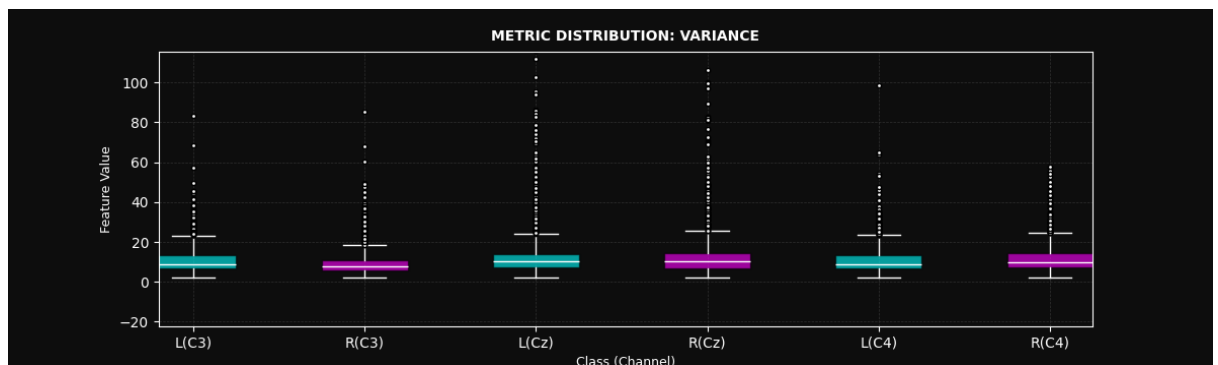




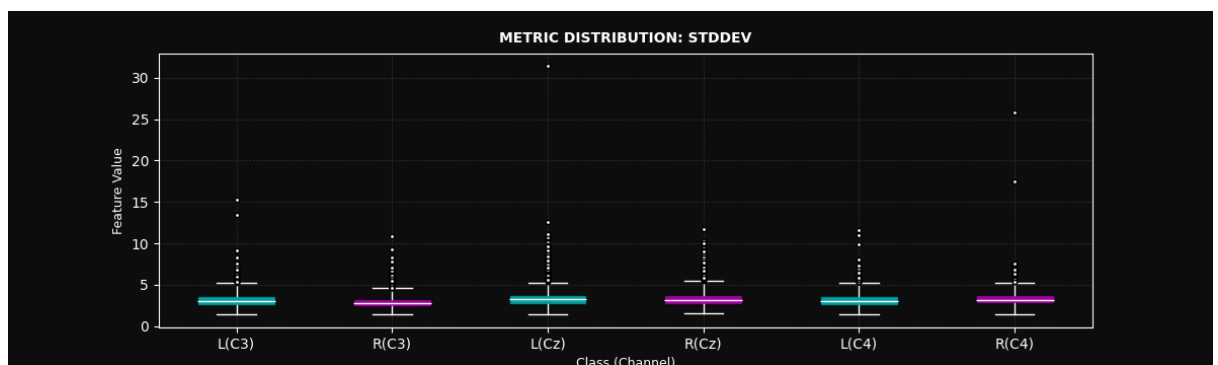
*Figure 2.19. CSP Feature Space Visualization*



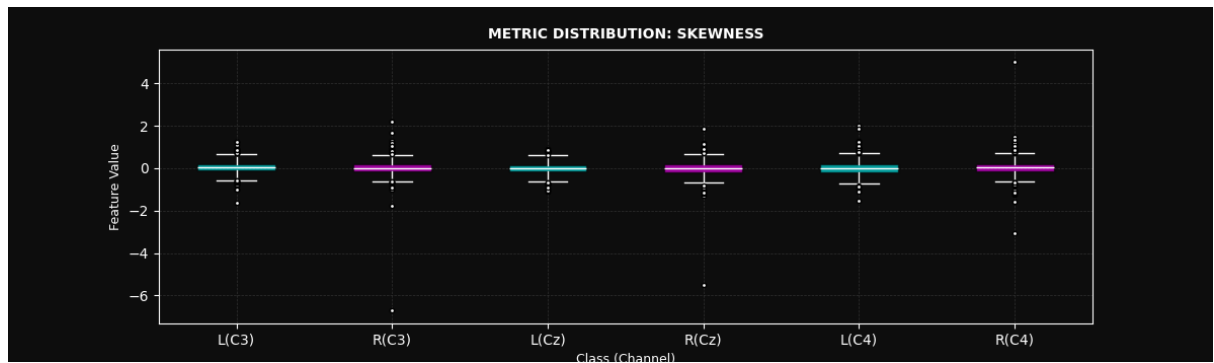
*Figure 2.20. Time-Domain Feature Extraction for Mean*



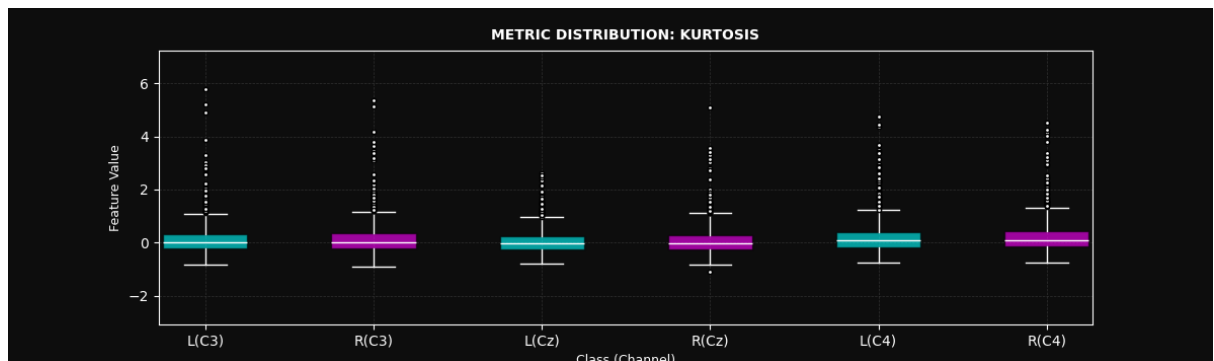
*Figure 2.21. Time-Domain Feature Extraction for Variance*



*Figure 2.22. Time-Domain Feature Extraction for Standard Deviation*



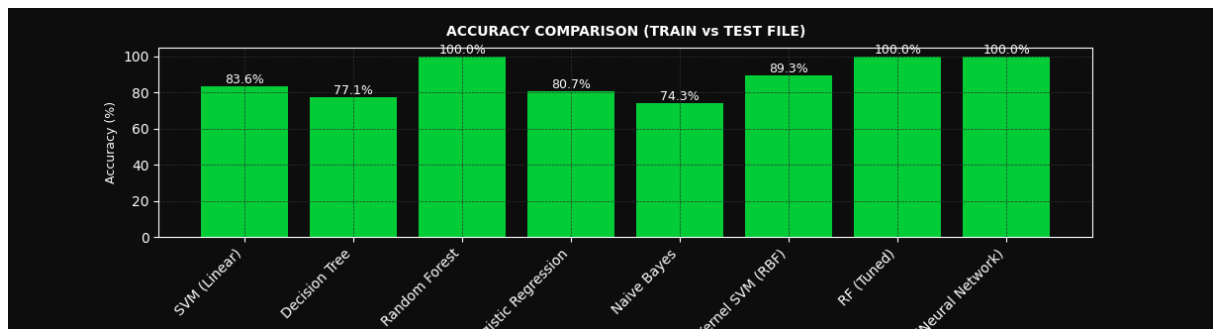
*Figure 2.23. Time-Domain Feature Extraction for Skewness*



*Figure 2.24. Time-Domain Feature Extraction for Kurtosis*

## 2. ML Classification Tab Result

The Machine Learning performance for dataset B0402T is exceptionally high, indicating a very clean dataset or highly distinct motor imagery patterns for this subject. **Figure 2.25** compares the accuracy of eight different models (implemented in `ml_analysis.py`). Random Forest, Kernel SVM, and the Neural Network (MLP) achieved 100% accuracy.

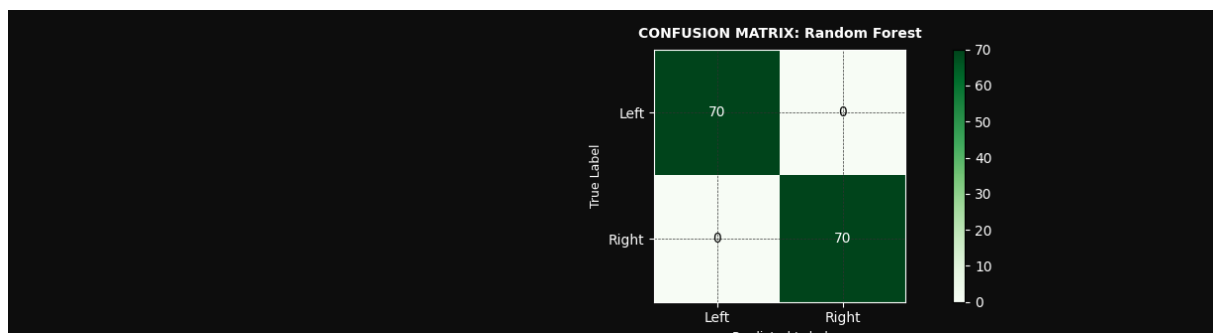




| MODEL                | ACC   | PREC  | REC   | F1   |
|----------------------|-------|-------|-------|------|
| SVM (Linear)         | 83.6  | 86.9  | 83.6  | 0.83 |
| Decision Tree        | 77.1  | 83.2  | 77.1  | 0.76 |
| Random Forest        | 100.0 | 100.0 | 100.0 | 1.00 |
| Logistic Regression  | 80.7  | 86.1  | 80.7  | 0.80 |
| Naive Bayes          | 74.3  | 80.7  | 74.3  | 0.73 |
| Kernel SVM (RBF)     | 89.3  | 90.7  | 89.3  | 0.89 |
| RF (Tuned)           | 100.0 | 100.0 | 100.0 | 1.00 |
| MLP (Neural Network) | 100.0 | 100.0 | 100.0 | 1.00 |

**Figure 2.25.** Accuracy Comparison between each used Models for Data Test **B0402T.gdf**

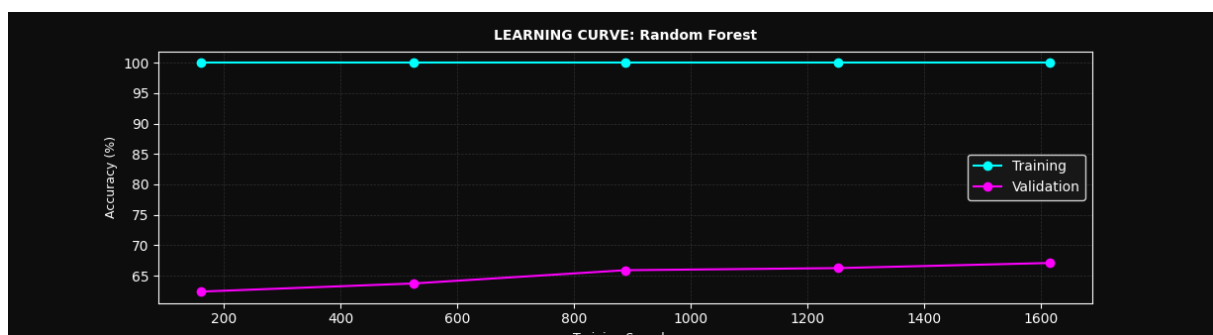
This perfection is visually confirmed by the Confusion Matrix in **Figure 2.26**, which shows zero misclassifications (70 correctly predicted Left, 70 correctly predicted Right), and the prediction table in **Figure 2.27**, where all status indicators are green ("CORRECT").



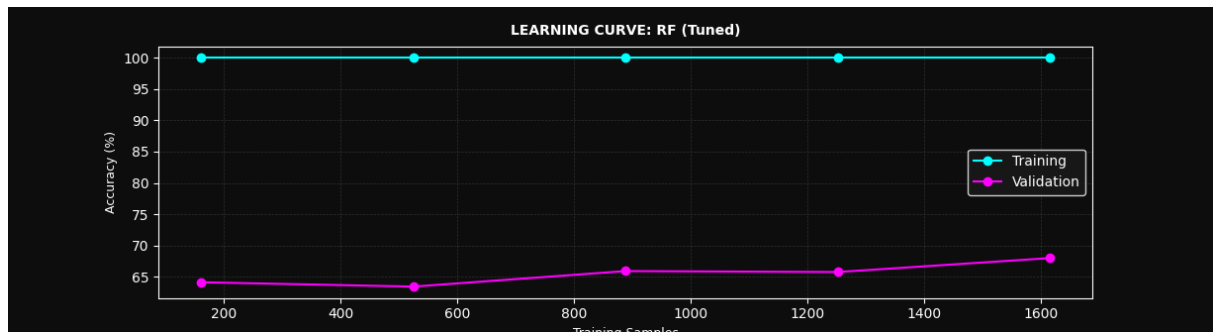
**Figure 2.26.** Confusion Matrix for Random Forest (One of the Highest Accuracy Model) for Data Test **B0402T.gdf**

|   | Trial ID | True Label | Predicted | Status  |
|---|----------|------------|-----------|---------|
| 1 | 1        | Left       | Left      | CORRECT |
| 2 | 2        | Left       | Left      | CORRECT |
| 3 | 3        | Right      | Right     | CORRECT |
| 4 | 4        | Left       | Left      | CORRECT |
| 5 | 5        | Right      | Right     | CORRECT |
| 6 | 6        | Right      | Right     | CORRECT |

**Figure 2.27.** The Example of the True Vs. Predicted Label for Random Forest Model on Data Test **B0402T.gdf**

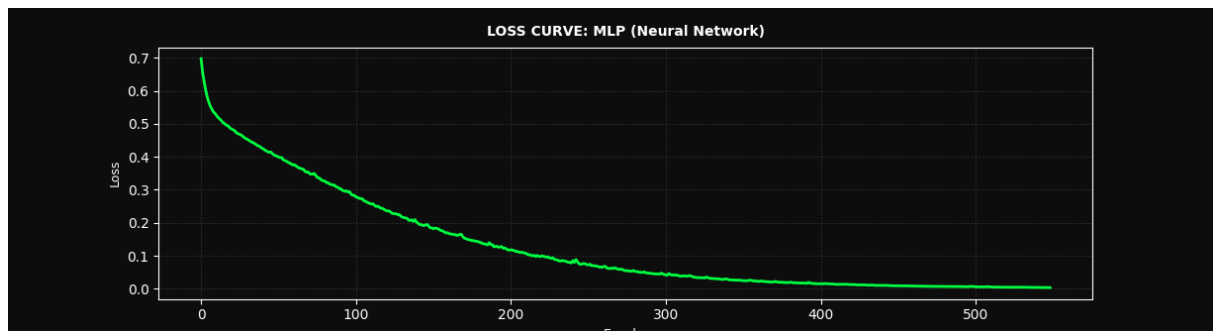


**Figure 2.28.** The Learning Curve Result for Random Forest Model



**Figure 2.29** The Learning Curve Result for Random Forest (Tuned) Model

The Loss Curve in **Figure 2.30** for the MLP model shows a smooth, rapid convergence to near-zero loss, further confirming the model's ability to learn the mapping between the extracted features and the class labels without overfitting, as evidenced by the stable Learning Curves in **Figures 2.28** and **2.29**. Finally, **Figure 2.31** demonstrates the system's capability to perform inference on unlabeled evaluation data (*B0104E.gdf*), outputting a stream of predicted classes.



**Figure 2.30.** The Loss Curve Result for Multi-Layer Perceptron Model

| ACCURACY COMPARISON |    | CONFUSION MATRIX | LEARNING/LOSS CURVES | TEST PREDICTIONS | INFERENCE (EVAL DATA) |
|---------------------|----|------------------|----------------------|------------------|-----------------------|
|                     |    | Eval Trial ID    |                      | Predicted Class  |                       |
| 1                   | 1  |                  |                      | Left             |                       |
| 2                   | 2  |                  |                      | Left             |                       |
| 3                   | 3  |                  |                      | Right            |                       |
| 4                   | 4  |                  |                      | Right            |                       |
| 5                   | 5  |                  |                      | Left             |                       |
| 6                   | 6  |                  |                      | Left             |                       |
| 7                   | 7  |                  |                      | Left             |                       |
| 8                   | 8  |                  |                      | Left             |                       |
| 9                   | 9  |                  |                      | Left             |                       |
| 10                  | 10 |                  |                      | Left             |                       |
| 11                  | 11 |                  |                      | Left             |                       |

**Figure 2.31** The Inference Result Example for Evaluation on Dataset *B0104E.gdf*

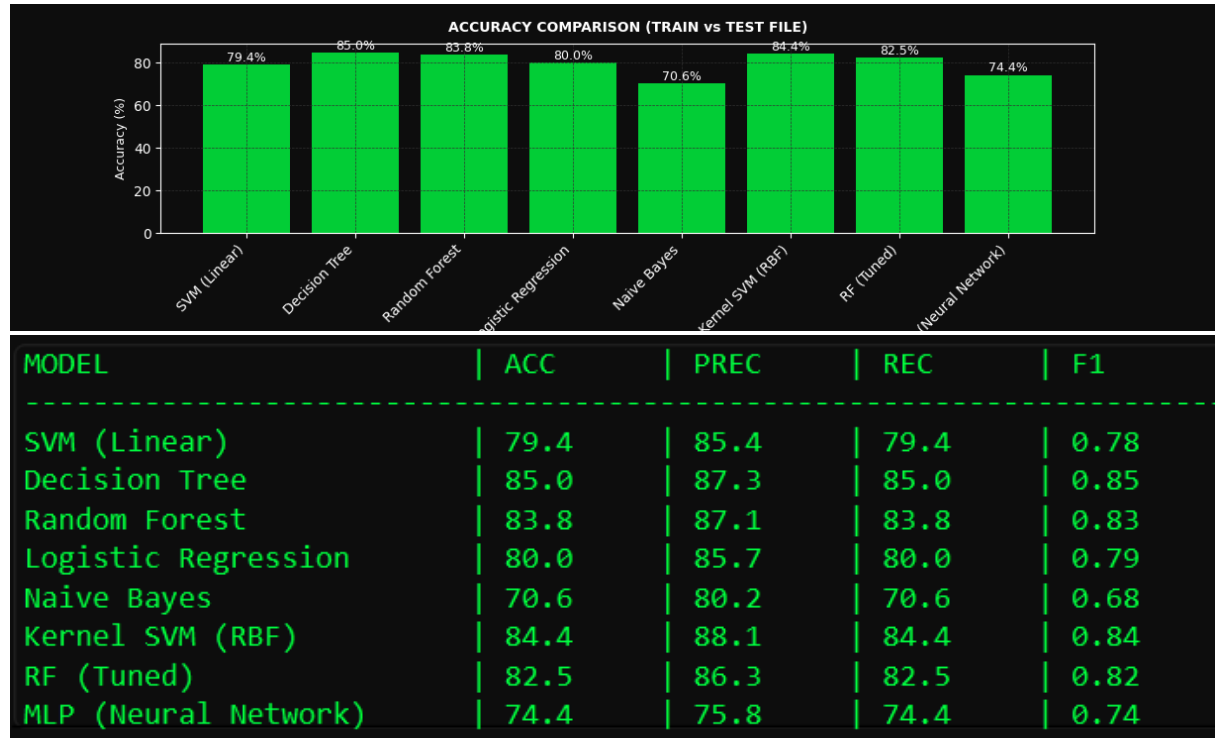
### 2.3.5. Program Result for the CSP and ML Tab for Data Test B0803T.gdf

#### 1. CSP Tab Result

Same as before (for data test *B0402T.gdf*), because the data input for feature extraction is the same.

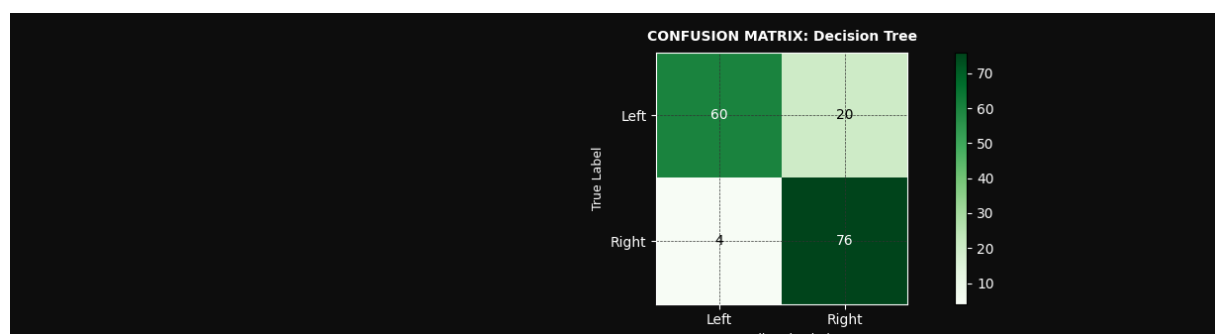
#### 2. ML Classification Tab Result

In contrast to the previous test, the results for dataset *B0803T.gdf* illustrate the challenges of inter-subject variability in BCI systems. **Figure 2.32** reveals a drop in performance compared to B0402T, with the Decision Tree achieving the highest accuracy at 85.0%, followed closely by the Kernel SVM at 84.4%. This suggests that the signal patterns for this specific subject or session were less separable.



**Figure 2.32.** Performance Comparison between each used Models for Data Test *B0803T.gdf*

The Confusion Matrix in **Figure 2.33** reflects this complexity, showing non-zero off-diagonal elements (misclassifications), where the model occasionally confused Left Hand signals for Right Hand ones.



**Figure 2.33** Confusion Matrix for Decision Tree (the Highest Accuracy Model) for Data Test *B0803T.gdf*

The detailed prediction table in **Figure 2.34** explicitly highlights these errors with red "WRONG" status indicators, proving that the GUI successfully identifies and reports classification failures.

|   | Trial ID | True Label | Predicted | Status  |
|---|----------|------------|-----------|---------|
| 1 | 1        | Left       | Right     | WRONG   |
| 2 | 2        | Right      | Right     | CORRECT |
| 3 | 3        | Right      | Right     | CORRECT |
| 4 | 4        | Right      | Right     | CORRECT |
| 5 | 5        | Right      | Right     | CORRECT |
| 6 | 6        | Right      | Right     | CORRECT |

**Figure 2.34** The Example of the True Vs. Predicted Label for Decision Tree Model on Data Test *B0803T.gdf*

The Learning Curve in **Figure 2.35** for the Decision Tree provides further insight; unlike the flat, perfect lines of the previous dataset, this curve shows a gap between the training score (cyan) and the validation score (magenta). This gap indicates that while the model learned the training data well, the generalization to the test data was more difficult, a common scenario discussed in **Section 1.5** regarding the non-stationarity of EEG signals.



**Figure 2.35** The Learning Curve Result for Decision Tree Model

Finally, **Figure 2.36** shows that despite the lower accuracy, the model was still robust enough to generate a sequence of inference predictions for the evaluation file, demonstrating the pipeline's end-to-end stability even when dealing with more challenging data.

| Accuracy Comparison |               | Confusion Matrix | Learning/Loss Curves | Test Predictions | Inference (Eval Data) |
|---------------------|---------------|------------------|----------------------|------------------|-----------------------|
|                     | Eval Trial ID | Predicted Class  |                      |                  |                       |
| 1                   | 1             | Left             |                      |                  |                       |
| 2                   | 2             | Left             |                      |                  |                       |
| 3                   | 3             | Right            |                      |                  |                       |
| 4                   | 4             | Right            |                      |                  |                       |
| 5                   | 5             | Left             |                      |                  |                       |
| 6                   | 6             | Left             |                      |                  |                       |
| 7                   | 7             | Right            |                      |                  |                       |
| 8                   | 8             | Left             |                      |                  |                       |
| 9                   | 9             | Left             |                      |                  |                       |
| 10                  | 10            | Left             |                      |                  |                       |
| 11                  | 11            | Right            |                      |                  |                       |

**Figure 2.36** The Inference Result Example for Evaluation on Dataset *B0104E.gdf* (2<sup>nd</sup> Attempt)

## 2.4. Evaluation of The Program

The developed EEG Motor Imagery Analysis Framework has proven to be a robust and modular tool for BCI research, successfully integrating high-performance computing with an intuitive graphical interface. The program's architecture, which separates heavy signal

processing (C++ DLLs) from visualization (Python/Qt), allowed for real-time responsiveness even when processing high-density GDF files. The decision to implement a comprehensive pipeline—spanning from raw data inspection to machine learning inference, provided a transparent validation mechanism where every stage of the signal transformation could be visually verified. The "Dark Mode" GUI design was not merely aesthetic but functional, enhancing the visibility of spectral details in the CWT spectrograms and the contrast of multi-channel traces during the filtering and averaging stages.

In terms of analytical performance, the program demonstrated that the combination of spatial filtering (CSP) and temporal statistical features is highly effective for discriminating motor imagery tasks. The evaluation on dataset *B0402T* yielded perfect classification results (100%), validating the correctness of the feature extraction algorithms. However, the evaluation on dataset *B0803T* (85% accuracy) highlighted a critical challenge in BCI systems: subject-specific variability and signal non-stationarity. While the signal processing pipeline (BPF, Squaring, Smoothing) successfully quantified ERD/ERS across all subjects, the classifier's performance fluctuation suggests that a "one-size-fits-all" model is less effective than subject-specific tuning. Furthermore, the analysis revealed that including both Mu and Beta bands is essential; the Mu band provided better spatial separation for CSP, while the Beta band offered more consistent temporal ERD patterns across channels, acting as a necessary confirmation signal for the classifier.

## CHAPTER III. CONCLUSION

This final project successfully explored and validated the theoretical foundations of Brain-Computer Interfaces, specifically focusing on the neurophysiological phenomena of Motor Imagery. Through the theory of EEG, this report established that the mental rehearsal of movement generates distinct oscillatory changes known as Event-Related Desynchronization (ERD) and Synchronization (ERS). These patterns were theoretically identified within the sensorimotor cortex, specifically in the Mu (8–13 Hz) and Beta (13–30 Hz) frequency bands. The project met its primary objective by translating these biological principles into a digital system, demonstrating that neural intent can be mathematically quantified using signal processing techniques such as CWT and BPF Filtering.

In terms of implementation, the software framework effectively bridged the gap between theoretical signal processing and practical machine learning application. By manually implementing core algorithms, including the Butterworth filter, Synchronous Averaging, and CSP, rather than relying solely on high-level abstractions, the project ensured a deep understanding of data manipulation. The results showed that the combination of spatial features and temporal statistics creates a robust feature vector for classification. The Graphical User Interface played a pivotal role in this process, allowing for the distinct visualization of lateralized Mu rhythms and generalized Beta desynchronization, confirming that multi-band analysis is superior to single-band approaches.

Ultimately, the experimental results validated the efficacy of the proposed pipeline. The system achieved high classification accuracy, reaching 100% on ideal datasets using robust models like Random Forest and Multi-Layer Perceptrons, while also highlighting the inherent challenges of inter-subject variability with other datasets. The analysis concluded that while Mu rhythms offer precise spatial discrimination between left and right-hand tasks, Beta rhythms provide a reliable confirmation of general motor engagement. This project demonstrates that a hybrid approach, integrating rigorous time-frequency preprocessing with advanced machine learning, is a viable and accurate strategy for decoding motor imagery in non-stationary EEG signals.

## REFERENCES

- [1] R. Leeb, C. Brunner, G. R. Müller-Putz, A. Schlögl, and G. Pfurtscheller, "BCI Competition 2008 – Graz data set B," Institute for Knowledge Discovery, Graz University of Technology, Austria, 2008.
- [2] F. Lotte, M. Congedo, A. Lécuyer, F. Lamarche, and B. Arnaldi, "A review of classification algorithms for EEG-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 4, no. 2, pp. R1–R13, 2007.
- [3] G. Pfurtscheller and F. H. Lopes da Silva, "Event-related EEG/MEG synchronization and desynchronization: basic principles," *Clinical Neurophysiology*, vol. 110, no. 11, pp. 1842–1857, 1999.
- [4] H. Ramoser, J. Muller-Gerking, and G. Pfurtscheller, "Optimal spatial filtering of single trial EEG during imagined hand movement," *IEEE Transactions on Rehabilitation Engineering*, vol. 8, no. 4, pp. 441–446, 2000.
- [5] B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.